Journal of Intelligent & Robotic Systems (2022) 105: 2
https://doi.org/10.1007/s10846-022-01582-8

SURVEY PAPER

Camera, LiDAR and Multi-modal SLAM Systems for Autonomous Ground Vehicles: a Survey

Mohammed Chghaf[1] · Sergio Rodriguez[1] · Abdelhafid El Ouardi[1]

Received: 14 April 2021 / Accepted: 24 January 2022 / Published online: 20 April 2022
© The Author(s), under exclusive licence to Springer Nature B.V. 2022

Abstract
Simultaneous Localization and Mapping (SLAM) have been widely studied over the last years for autonomous vehicles. SLAM achieves its purpose by constructing a map of the unknown environment while keeping track of the location. A major challenge, which is paramount during the design of SLAM systems, lies in the efficient use of onboard sensors to perceive the environment. The most widely applied algorithms are camera-based SLAM and LiDAR-based SLAM. Recent research focuses on the fusion of camera-based and LiDAR-based frameworks that show promising results. In this paper, we present a study of commonly used sensors and the fundamental theories behind SLAM algorithms. The study then presents the hardware architectures used to process these algorithms and the performance obtained when possible. Secondly, we highlight state-of-the-art methodologies in each modality and in the multi-modal framework. A brief comparison followed by future challenges is then underlined. Additionally, we provide insights to possible fusion approaches that can increase the robustness and accuracy of modern SLAM algorithms; hence allowing the hardware-software co-design of embedded systems taking into account the algorithmic complexity and the embedded architectures and real-time constraints.

Keywords SLAM · Camera · LiDAR · Localization · Mapping · Fusion · Hardware systems

1 Introduction

Simultaneous Localization and Mapping (SLAM) continues to dominate a large share of the research conducted in the robotics community. SLAM is the process that enables a robot to answer two questions: Where am I? and what does the surrounding environment look like? These two questions being dependent on each other, complications arise when trying to answer both of them simultaneously. This is the key distinction between odometry and SLAM. Indeed, odometry's primary focus is local consistency and the continuous estimation of the trajectory and the poses of the vehicle. It also handles the local optimization of a small set of poses. On the other hand, SLAM seeks to map the surroundings while achieving a globally consistent trajectory estimation by combining information about loop closures with ego-motion measures. Since SLAM has to assure loop detection and relocalization capabilities, building a consistent map in parallel is necessary. SLAM is otherwise reduced to odometry when the loop closure module is absent from the framework.

Several SLAM algorithms have been developed to solve the mapping and localization problem. The type of sensors employed and the mathematical model applied are the main characteristics that classify these algorithms. Multiple frameworks have been proposed to simultaneously estimate the position of the vehicle navigating in an unknown environment while gradually constructing a map of it using the various sensors mounted on the vehicle, such as cameras, LiDAR, Radar, GNSS receiver, and IMU, to name a few. This field of research has attracted wide attention especially for its broad application to navigating robots (underground, underwater or spatial), unmanned aerial vehicles, indoor service robots, Virtual/Augmented Reality systems or autonomous vehicles. This latter application,

✉ Mohammed Chghaf
mohammed.chghaf@universite-paris-saclay.fr

Sergio Rodriguez
sergio.rodriguez@universite-paris-saclay.fr

Abdelhafid El Ouardi
abdelhafid.elouardi@universite-paris-saclay.fr

[1] Université Paris-Saclay, ENS Paris-Saclay, CNRS, SATIE, 91190, Gif-sur-Yvette, France

being one of the most challenging, is the main scope of our article. Indeed, autonomous vehicles engender additional complexity related to real-time constraints, the high dynamics of the scene, the large-scale of the surrounding environment and the long term variations that can occurs in a visited place.

The complex problem introduced by SLAM can be addressed by integrating multiple sensors in a single framework that can offer the level of integrity and redundancy that a SLAM system for autonomous vehicles needs. However, in literature, many existing solutions focus on improving established ones, while fewer approaches are groundbreaking. Historically, one of the most relevant solutions was to use fusion techniques to enhance accuracy. Following these same steps, it should be noted that one of the most creative techniques in SLAM were the ones that combined several modalities even if they are very different from each other and can induce some complications. Indeed, with each sensor having its own strengths and weaknesses, formulating a solution that can integrate a number of them is shown to improve the overall robustness and accuracy.

Moreover, most of the state-of-the-art solutions focus on the improvement of the algorithmic aspect of the SLAM problem. Even if this approach is very essential to the progress of research in the robotic community, much progress remains to be done to study the design of specific, embedded architectures and real-time systems. The hardware constraints associated with SLAM for autonomous vehicles (small design, low-consumption, near-sensor processing and real-time data processing) require researchers to adopt a systemic approach in the resolution of this problem. Defining an efficient distribution of the computational load over general purpose processing units and application-specific hardware accelerators is still an open challenge that is not explored enough despite its many promising possibilities.

Our aim in this paper is to highlight the novel groundbreaking existing SLAM strategies with a special focus on the solutions based on both Camera and LiDAR, while exposing existing problems and future possible solutions for automotive applications evolving in a large-scale dynamic environment. In contrast to existing surveys such as [27] that limit their scope to the algorithmic aspect of SLAM and omitting the desired application of such system, we present a more complete survey taking into account a systemic approach. Our paper, present a detailed description of both software and hardware coupled camera-LiDAR SLAM systems applied to autonomous vehicles. Moreover, we provide the reader with more insights to actual trends and future research orientations to SLAM systems based on sensor fusion. The main contributions of this work are:

1. Reviewing, discussing, and comparing the commonly used sensors in the SLAM framework.
2. Deeply analyzing state-of-the-art SLAM systems depending on the general tendency of the use-case, the type of the adopted scenario, the estimated precision and most importantly the corresponding hardware architecture.
3. Reviewing the most recent SLAM systems depending on the sensor and software technologies used in the estimation and optimization steps.
4. Presenting a simple and comprehensive survey of SLAM frameworks using multi-modal sensors and classifying these solutions depending on the coupling strategy.
5. Concluding on current developments and proposing future direction of research on multi-modal SLAM and its hardware/software optimization to design embedded real-time systems.

The remainder of this paper is structured into five sections organized as follows: Section 2 presents the main sensors used in the case of autonomous vehicles, details about their particularities and a justification of the wide choice of LiDAR and Camera is added. Section 3 is a brief memo on SLAM formulation, its main building blocks and the classification of techniques used to solve it. Section 4 reviews the main mono-modal SLAMs in the literature that use either camera or LiDAR sensor in their process, a description of each approach followed by a comparison and the future challenges to overcome is provided. Section 5 presents and surveys different state-of-the-art multi-modal strategies using both camera and LiDAR at least, a classification of these strategies is established. Section 6 discusses the future challenges SLAM systems still face and introduces possible research directions to solve some of them. Figure 1 gives the general structure of the paper and allows the reader to target each SLAM method and the different levels of data processing in a SLAM system.

2 Multi-Sensor Data Fusion in the SLAM Framework

As mentioned in the introduction (see Section 1), the complexity of SLAM applied to autonomous vehicles is a result of the duality of the problem that needs to simultaneously localize itself and map the surrounding environment. Therefore, using data sources that can provide both abundant amount and reliable information is critical. The commonly used sensors can be classified as proprioceptive and exteroceptive sensors. The first category of sensors measures the state of the vehicle itself (position, speed of the wheels, etc.) which can be exploited only to acquire pertinent information

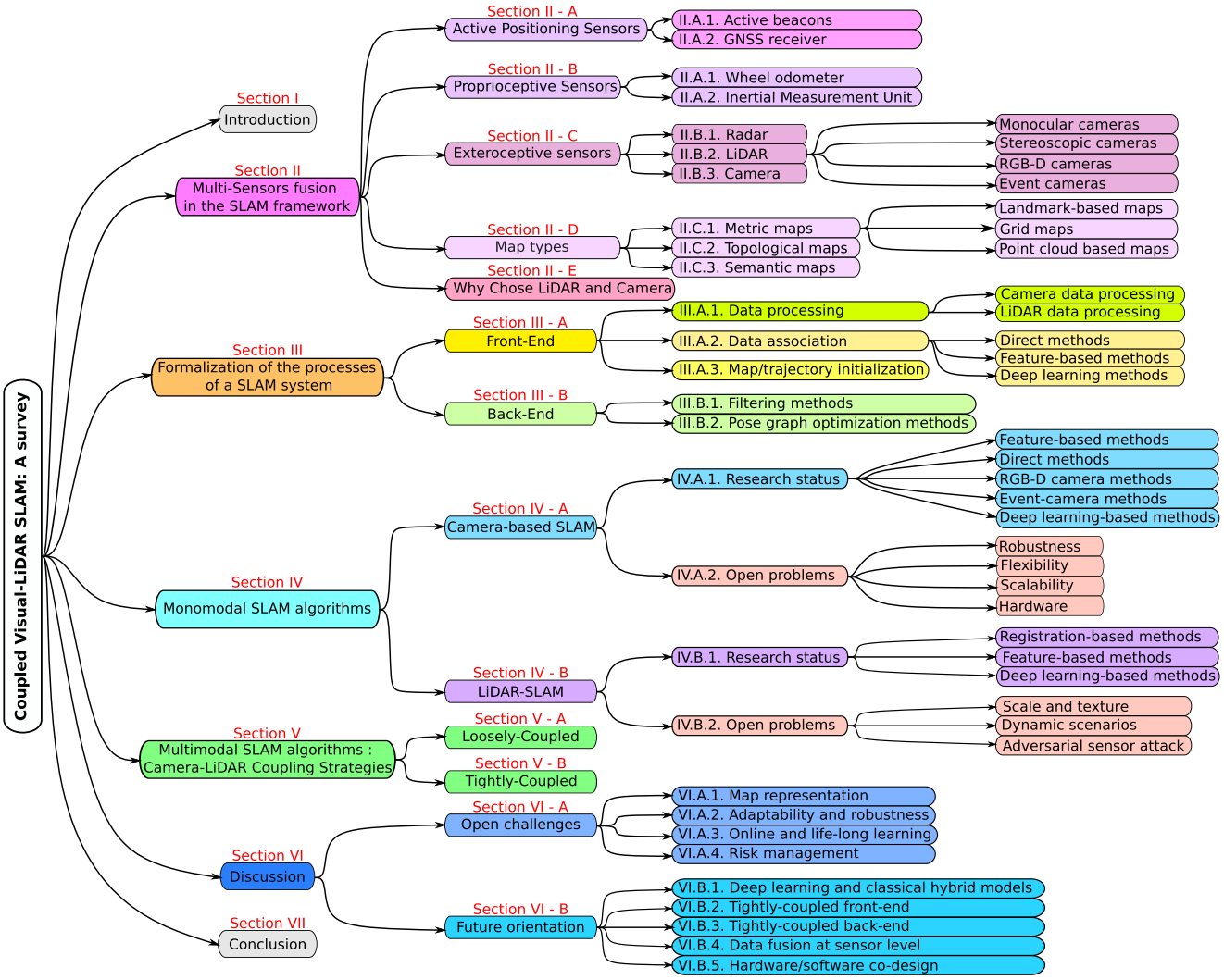


Fig. 1 Schematic outline of the structure of the paper

about the carrier localization. Fortunately, the second category is dual and can answer the need of SLAM by providing information that can ensure obtaining both localization and mapping by measuring the state of the environment surrounding the carrier (temperature, range, luminosity, etc.). However, proprioceptive sensors become a necessity for a reliable system in case exteroceptive sensors fail. For instance, this is the case when the surrounding environment is highly dynamic and the sensor reaches the data rate limits, preventing the system from locating itself.

Since the conventional sensors that will be reviewed are able to generate different kinds of information, the type and quality of derived maps can vary greatly. The rest of this section will be dedicated to the discussion of the most common sensors in the context of autonomous vehicles and the most commonly used map representations.

2.1 Active Positioning Sensors

2.1.1 Active Beacons

Active beacon navigation systems are commonly used in ships and airplanes, as well as in indoor mobile robot systems. They can be detected reliably and provide accurate positioning information with minimal processing. Two different types of active beacon systems can be distinguished: trilateration (distance measurements) and triangulation (angle measurements). Practical applications commonly use three or more active beacons in the environment and the accurate mounting of beacons is required for accurate positioning. The position of a mobile robot in a three dimensional space can be calculated basically from the distance information from the beacons which have their own

absolute position information. This approach allows high sampling rates and yields high reliability, but it does also incur high cost in installation and maintenance [14].

2.1.2 GNSS Receiver

The Global Navigation Satellite System (GNSS) is the most intuitive solution to provide Geo-referenced position of the vehicle. The concept is based on specialized satellites that continuously transmit a radio signal containing the current time and data about their position. Each GNSS receiver requires at least four satellites to compute four unknown quantities (three position coordinates and the shift of its clock relative to the time reference in GNSS). This condition challenges the consistency of this sensor in a multi-path environment and during signal outages which limits its use to outdoor environments where the signal quality can not be disturbed by high buildings or dense trees [74]. Moreover, the vehicle's orientation can be determined by measuring the relative positions of multiple GNSS antennas mounted on different positions. However, such solutions could be challenging to integrate in automotive applications considering dimensions constraints and the minimum baseline between two antennas (40cm to 3m).

Additionally, it should be noted that unlike proprioceptive sensors that only allow determining relative motion, active positioning sensors (like GNSS receivers, active beacons, etc.) are able to provide the absolute position of their carrier.

2.2 Proprioceptive Sensors

2.2.1 Wheel Odometer

The wheels of terrestrial vehicles have encoders that make it possible to count the number of the rotations completed by the wheels and thus to estimate the distance traveled. It is then possible to estimate the position and orientation of the vehicle relative to its starting point. Wheel odometry exhibits a drift which increases during movement. This drift comes in particular from the loss of grip of the wheel due to sliding, it is therefore quite variable depending on the nature of the ground (smooth or rough ground) and the trajectory followed by the vehicle (rather straight trajectory or with many turns).

2.2.2 Inertial Measurement Unit

An Inertial Measurement Unit (IMU) is made up of three accelerometers and three gyrometers. Accelerometers measure linear acceleration in the three directions of space x, y, and z. Gyrometers are used to obtain angular speeds around the three axes of rotation (roll, pitch and yaw). The IMU combines information from motion model and temporal measurements integration to infer the position (x, y, z) and the orientation $(\theta_x, \theta_y, \theta_z)$ relative to the previous position from these six measurements [72]. The estimation step is usually obtained by a filter, such as the Kalman filter, which provides an estimate of position, orientation and speed. The localization is performed using an integration method, while the calculation of the position and the orientation with respect to the point of origin is carried out by combining the displacements calculated step by step during the movement. This sensor exhibit a drift which increases rapidly during movement due to the integration step of the measurements required to calculate the position. Therefore, it can not be used alone to perform an accurate pose estimation.

2.3 Exteroceptive Sensors

2.3.1 Radar

A Radar (an acronym for Radio Detection And Ranging) is a measuring instrument that indicates the presence of a distant object, its size, its speed, and its direction. It can operate in challenging weather conditions such as fog, rain, snow, and low/high luminosity.

In general, two radar technologies are used: Pulse-Doppler and Frequency-Modulated Continuous-Wave (FMCW). The Pulse-Doppler technology uses a transmitter that emits radio waves and a receiver that collects the reflected waves to accurately measure the object's velocity by detecting changes in the reflected wavelengths. Alternatively, FMCW technology, which is commonly used in the automotive context, is based on continuously transmitting a carrier frequency and extracting useful information from the return echo such as the round trip delay of received signal or the frequency difference between the transmitted and received signals [49].

Consequently, object detection accuracy depends on the object's size, the distance at which it is located, its absorption characteristics and the reflection angle. In general, radars with a large opening angle have a short range and vice-versa.

Generally, radar sensors cannot distinguish between a stationary target and unwanted echoes [52]. Existing techniques tend to use additional processing to reduce the measurement errors. Plus, current automotive radar sensors make relatively few detections in one sampling loop due to the limited sensor angle resolution [95]. Most existing methods that perform object recognition using radars use beat frequencies in the frequency domain. Other recent research analyzes the characteristics of the radar signals of stationary and moving objects, and maps stationary and moving objects to the time-frequency spectrum [114].

Current radar sensors are usually noisy due to the false-positive, false-negative or the range and angular errors. The users can either wield the manufacturer's software to reduce these noises or choose to handle the raw sensor data. In the first case, the provided software usually do not specify additional information on the processing methods the level of uncertainty associated with the processed data. In the second case, working on raw sensor data directly does not solve the issue. In fact, radar is more prone to noise than LiDAR and camera since it is highly sensitive to the surface reflectance and the reflector pose. Moreover, multipath interferences can lead to more noises and outliers since it can introduce inconsistency between consecutive frames, resulting in many false alarms [49, 104].

2.3.2 LiDAR

A LiDAR (an acronym for Light Detection And Ranging) relies on laser beams to determine object's position. The sensor sends laser pulses towards the surrounding world and obtains the distance of the scanning points from its center by the time-of-flight method. The high frequency, the high precision, the long range and the absence of illumination influence are all known advantages of this technology. Moreover, LiDAR reflective measurements provide more additional information on the structure of the scene. Thus, LiDARs can be used in indoor or outdoor environment and can reproduce their 2D or 3D representations. LiDARs have also higher resolutions than radar due to the large number of scans and the high density of scanning points.

Unfortunately, LiDARs can not measure velocity directly, therefore the user needs to rely on successive scans to estimate it [54, 89]. Additionally, the weather has a more significant impact on LiDARs than radars.

2.3.3 Camera

The camera is the most used sensor in SLAM systems and the most studied by researchers. It is mainly used to achieve tasks such as target identification and tracking, objects detection or environment map building. There exist different types of cameras:

Monocular Cameras Historically, in case of a single camera, scene reconstruction requires sensor movement in order to detect and track salient spots using image processing techniques to match the same spots in the previous frames. However, some emerging techniques using Artificial Intelligence aim to estimate depth from a single color image [127]. These matched and tracked features provide information about the vehicle ego-motion which can be translated into trajectory. The main drawback of monocular cameras is the scale factor drift while calculating distances between points.

Stereoscopic Cameras This sensor consists of two monocular cameras, located at a fixed and known distance with overlapping fields of view. With stereo camera, scene depth information is estimated through matching and triangulation. However, the baseline between the two monocular cameras and their resolution limit the maximum depth range. Therefore, the broader the baseline, the more precise depth can be estimated, albeit at the expense of the field of view in the overlapped area in both cameras. In [26], De Croce et al. presented a distributed architecture for the S-PTAM stereo SLAM system. This system is designed for mobile robots with limited processing capacity since it allows the localization module to operate onboard. In parallel, the mapping module is run on a remote base station. Thus, lowering the computational load of the embedded processor.

RGB-D Cameras These cameras have the advantage of providing pixel depth directly. They are able to supply 3D structure of the environment in addition to the texture information in real time. Moreover, the scale is known due to the availability of the acquired information in the metric space. This is accomplished by using stereo-vision or other technologies such as stereo imaging, structured-light or Time-of-Flight (ToF). RGB-D cameras can be described as the most complete visual sensor because they combine multiple vision properties (monocular and stereoscopic), and they add active support from light projectors [86]. They are also compact and can integrate an IMU as well. However, due to their limited range and their sensitivity to sunlight they are mostly used for indoor navigation.

Event Cameras Event-based cameras, also known as dynamic vision sensors (DVSs) are bio-inspired novel sensors that can provide asynchronous stream of "events", with microsecond resolution, upon changes in light intensity for each individual pixel. These streams are recorded with their positions, timestamps, and polarities continuously. Therefore, there is no notion of "frame" as in conventional cameras. Furthermore, compared to conventional image sensors, they offer significant advantages: high temporal resolution, high dynamic range, no motion blur, and much lower latency, bandwidth and power consumption [58, 123, 137]. However, event-camera based SLAM is still in its early stages of research.

2.4 Map Types

In [130], IEEE has released a standard for 2D map representation of environments. However, 3D maps require further

exploration since they present additional challenges by taking into account their capacity, performance and robustness particularly in dynamic, large scale and open environments. The most commonly used map representations are: metric maps, topological maps and semantic maps. It should be noted that metric maps have the highest level of geometric information while semantic maps have the highest level of abstraction. Topological and hybrid maps can achieve a certain level of balance between geometric information and abstraction.

2.4.1 Metric Maps

This category of maps focuses on the geometric properties of the environment. The more precise the metric map is, the more accurately the vehicle can localize itself. Unfortunately, precision come at the cost of memory and computation. Hence a trade-off has to be reached to maintain performances in terms of accuracy, quality, execution time, memory management and power consumption.

Landmark-Based Maps In order to meet the limited storage and computational resources, landmark-based maps were introduced. This kind of maps represents the environment as a set of 3D landmarks that have well-defined features and corresponding spatial coordinates. This discretization of the environment reduces the data volume of the obtained description, hence the name "sparse representation" (see the illustration of Fig. 2). SLAM systems such as [26, 86] use and build landmark-based maps under the assumption that the environment has distinctive features. Otherwise, the algorithm will fail.

Grid Maps The most common scheme of 3D grid maps is Octomap which is based on Octree [50]. The data structure is based on trees, where each node can be recursively subdivided into eight sub-nodes, as illustrated in Fig. 3. Leaf nodes of Octotree store the occupancy probability of spatial space. On one hand, this simplified representation facilitates merging information from different sensors by taking into account the sensor's reliability. On the other hand, data volume of 3D grid maps grows at a much higher rate than the scale of the mapped region. One possible solution to this problem is adaptive resolution where only interesting areas profit from increased resolution.

Point Cloud-Based Maps These dense representation of the environment is based directly on 3D point cloud obtained from the sensor (see Fig. 4). SLAM systems using such techniques [47, 88] provide rich environment information and presents easier mapping process. However, data is redundant in such representations and data volume become impractical in large-scale scenarios. In modern SLAM solutions, this map representation can be carried out only on local areas or offline for global maps in order to respect the real-time constraints of map building.

2.4.2 Topological Maps

A topological map can be seen as a graph where a node represents a certain place and the edges represent the paths betweens places. Each time the vehicle reaches a new area, a new node is added. This representation is highly efficient and compact. However, this solution is not easily scalable to describe large environments. Indeed, such

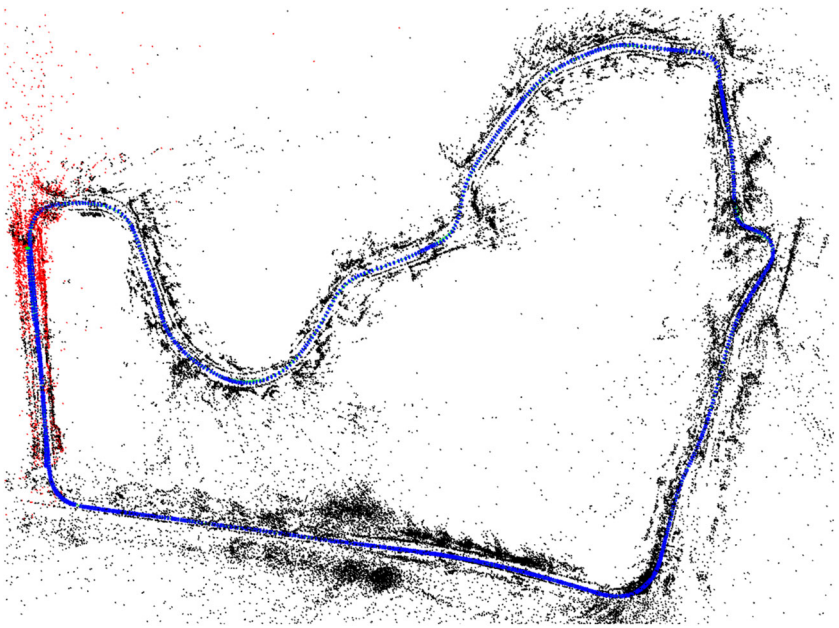


Fig. 2 Feature-based map of a rural area generated by [86]

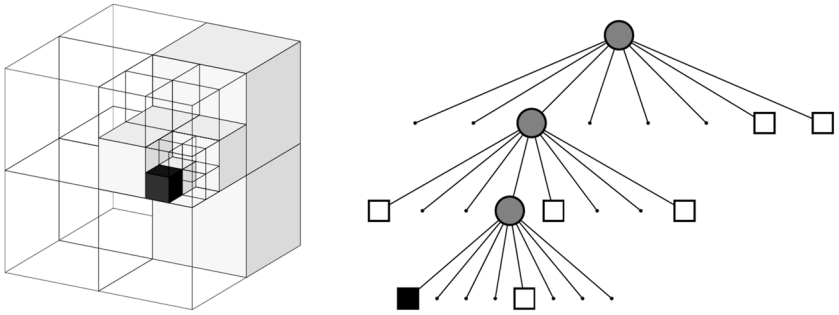


Fig. 3 Example of an Octree. A cell is colored in black when occupied and in shaded white if not. The left voxel shows the volumetric model and the model on the right is its corresponding tree representation [50]

minimalist description can lead to erroneous localization. Hybrid maps were built on topological maps by associating metric information [13] or semantic information [9] to local spaces. An example of hybrid map (metric-topological) is shown in Fig. 5.

2.4.3 Semantic Maps

In addition to spatial information, semantic maps provides higher level of abstraction than the other map representations. In fact, this kind of maps can facilitate more complex tasks for autonomous vehicles such as planning and navigation by combining the semantic and geometric information [32, 126]. Semantic elements can be either background classes (e.g., ground, vegetation, wall) or object classes (e.g., truck, person, building). Each semantic element can have several attributes and sub-elements (e.g., building can be of a certain color and have a certain number of doors and windows). Therefore, the relations in semantic maps are hierarchical and need large computational resources. Figure 6 shows an example of a semantic map with each class label represented with a different color. One of the main challenges that remain is the estimation of more fine-grained semantic information, such as lane structure or road type [20].

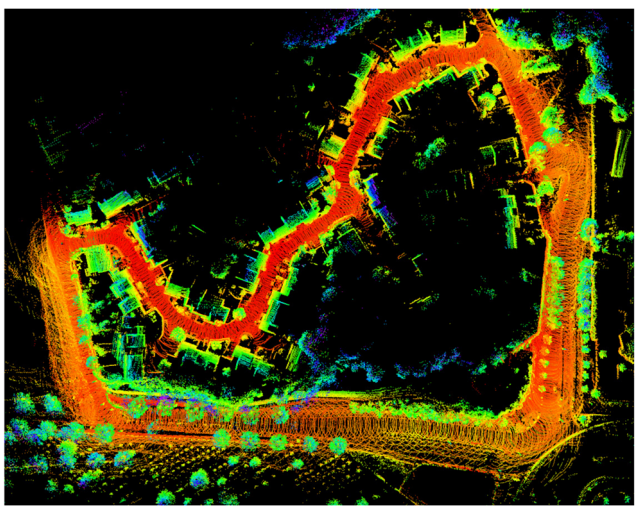


Fig. 4 Point cloud map generated by LeGO-LOAM [108]

2.5 Why Choose LiDAR and Camera

To develop an ultimate SLAM for automotive application the system must be equipped with sensors that can provide abundant information about the surrounding space all while being able to handle highly dynamic environment and large scale. The global framework has to be capable of obtaining the information from multiple streams of data to ensure reliability, redundancy and approach a fail-safe system. Therefore, using an RGB-D camera is a good choice due to its capacity to inherit from monocular and stereoscopic vision systems [47]. In fact, its properties are well-suited to SLAM for automotive application since it generates depth maps in addition to its stereo vision capabilities. The direct and rich measurements of the environment produced by this sensor can provide both the localization and the map. However, to deal with large scale, a sensor with long range is needed. This task can be achieved by using a LiDAR. Unfortunately, in highly dynamic scenarios exteroceptive sensors are no longer able to provide their localization and are limited by data processing rate. Consequently, this information can be retrieved by integrating an additional IMU to the system [7, 96, 123].

Indeed, cameras are inexpensive and energy-efficient, but they suffer from environmental variations due to several sensitivities such as low lightning conditions dramatic changes in light intensity or spurious detection. These sensors are also affected by other factors such as the lack of overlap between successive frames, texture-less images, motion blur, etc. Additionally, cameras remain limited in range. Moreover, they can only provide 2D representation of the world which induces information loss and requires additional computing power to map the world in 3D or to track and classify objects [1].

In the case of radars, range can be as short as a few meters or as much as a few hundred meters [49]. Although they are less effective at very short distances. Their major drawback on the other hand is their poor resolution and

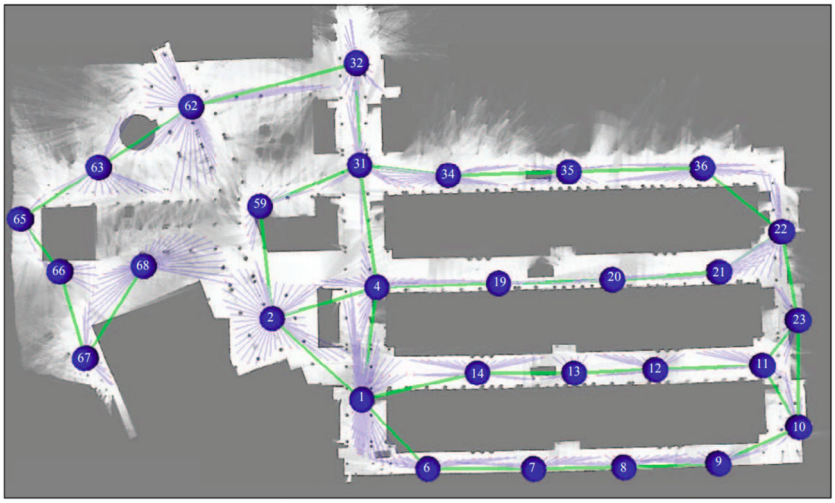


Fig. 5 Example of metric-topological map [13]

their spurious detection which makes them less accurate than cameras and LiDARs. Besides, they can not distinguish different targets or colors which are essential in modern SLAM applications where tracking and classifying dynamic objects is crucial [104]. Finally, radars are still not accurate enough to provide a reliable 3D representation of the environment, therefore, their coupling with visual solutions is more challenging [49].

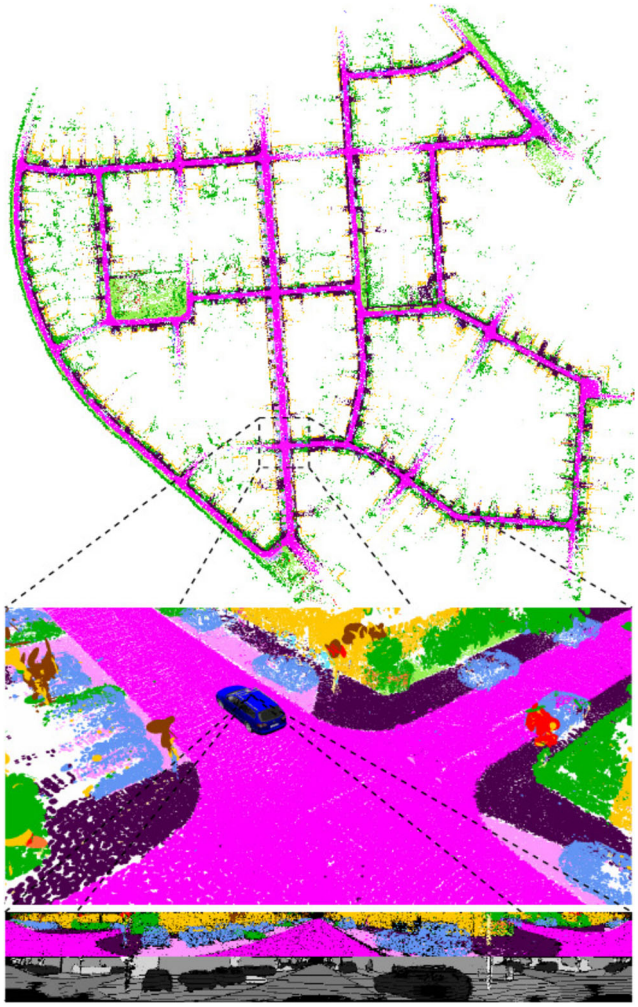


Fig. 6 Semantic map generated using LiDAR scans. Each color represents a class label [20]

Alternatively, LiDARs are commonly used in autonomous vehicles because of their wide field of view and long range plus their ability to provide direct access to 3D information. Moreover, in comparison with cameras, LiDARs have the advantage of measuring distances with the associated noise independently of the distance and the lightning conditions.

In addition to all these reasons, only camera and LiDAR sensors are able to produce all the map types presented above. Therefore, implementing a solution that can use a high volume map locally (e.g., semantic map) and a low volume map (e.g., topological map) globally can be easier using these both modalities. Authors coupled other modalities such as camera-inertial SLAM in [96, 107, 123], LiDAR-inertial SLAM in [7, 109], LiDAR SLAM aided by wheeled odometers in [18] or a fusion of GNSS and SLAM systems in [3, 41]. However, we believe that using only exteroceptive sensors (camera and LiDAR) is sufficient to provide an accurate localization and a reliable map representation while avoiding drift or masked areas introduced by proprioceptive sensors.

Finally, each sensor has its strengths and weaknesses as summarized in Table 1. Therefore, using multiple modalities to perceive the environment is a necessity to increase the reliability and accuracy (in terms of translational and rotational localization errors) of the system and to decrease its faultiness [42, 135, 139]. Beyond all the advantages mentioned above, cameras have the lead because they have been widely studied in recent years, they can provide large amounts of information about the environment and their low power consumption ability makes them more suited for embedded systems. Furthermore, LiDARs have other strong

Table 1 Comparison of exteroceptive sensors

Sensor	Advantages	Disadvantages	Output size (Mbps)	Power consumption (Watts)	Method	References	Map Scale / Density	Dynamic environment	Translational error
Radar	1. Applicable for all-weather 2. Long range	1. Impractical for static objects 2. False alarms	0.1 to 5	7 - 50	Feature	[31, 49, 104]	Medium / Sparse	Low	∼ 1m
					Direct	[8, 20, 29]	Large / Dense	Medium	Large: ∼5m Small: ∼4cm
LiDAR	1. Long range 2. Brightness independent	1. Limited by bad weather 2. High cost	2 to 20	8 - 60	Feature	[48, 54, 108, 134]	Large - Small / Dense	Low	Large: ∼5m Small: ∼4cm
					DL	[32, 66, 78, 122, 126]	Large / Dense	Medium - High	∼3m
					Feature	[17, 22, 25, 61, 72, 84, 86, 92, 93]	Large - Small / Sparse	Medium	Large: ∼10m Small: ∼15cm
	1. Easy object identification	1. Limited by bad weather			Direct	[21, 34–36, 91, 125]	Small - Medium / Dense	Low	Medium: ∼3m Small: ∼10cm
Camera	2. Abundant information	2. Luminosity dependent	60 to 400	0.1 - 13	RGBD	[47, 51, 65, 88, 96, 102]	Small / Dense	Low	∼12cm
	3. Low cost	3. Bad depth estimation			Event	[58, 123, 137]	Small / Sparse	Low	∼15cm
					DL	[5, 59, 80, 117, 129]	Small / Dense	Low	∼10cm

points like high accuracy, high refresh rate and the ability to detect high level of details [48, 108, 134]. For all these reasons, the combination of these two sensors is the main technological choice for many modern SLAM systems.

3 Formalization of the SLAM Process

Despite the different designs that can be found in literature depending on the individual integrated tasks, a SLAM process is generally composed of two essential blocks: front-end and back-end. These blocks aim to robustly solve real world challenging driving scenarios at a large scale. A conceptual diagram of this integrated SLAM process is shown in Fig. 7 showing the relationship of these blocks.

For the most part the front-end block's major purpose is to estimate the vehicle's pose given by its position and its orientation. This block has multiple tasks and relies on data gathered from different sensors to calculate the pose estimate. Due to external factors, such as weather phenomenon or internal sensor drifts, these measurements will inevitably carry additional noise that will generate increasing errors in the pose estimate over time. It is then the goal of the back-end block to mitigate these cumulative errors in order to improve the estimation accuracy by means of filtering or optimization algorithms. A more generic classification is that the first block is very sensor dependent, while the second block relies on an abstract representation of the data that creates a model of the environment. There is no explicit description of the limits of the two blocks due to its variations according to the sensors and algorithms used, however, in the scope of this work they will be formulated as described in the following subsections. This description is inspired by the most common camera-based SLAM components, since it is the most studied SLAM. In most cases, the back-end block provides feedback to the front-end block and is used in the data association step. This feedback is either a filter estimation in case of filtering methods or a pose/map refinement in pose graph optimization methods.

3.1 Front-End

This block's main task is to process sensor information and extract constraints from it that will be fed to the Back-End block. In order to better understand the Front-End block, this section details its common components with a brief description of their implementation. It should be noted first of all that this block implicitly assumes that the intrinsic parameters of the used sensors are known based on an offline calibration step. The Front-End often includes additional data processing modules that are essential to the preprocessing of sensor data such as temporal alignment, coordinate transformation, scaling, etc.

3.1.1 Data Processing

This task is the first step of the SLAM process and its goal is to interpret the raw information provided by the sensor as data that can help the autonomous vehicle infer the map structure and the trajectory. This operation is achievable by identifying distinguishable landmarks from the raw sensor data.

Camera Data Processing There are two approaches in order to process this data: feature-based and direct. On one hand, the direct approach either aims to work directly on every pixel level by generating a dense map as described in **DTAM** algorithm [88], or by firstly computing the areas that have significant intensity gradient before comparing with synthetic view images as in **LSD-SLAM** [35] or **DSO** [34, 125]. This strategy is time consuming and can limit the real-time performance. However, it allows working on raw data to avoid errors caused by the additional step of feature extraction. On the other hand, in the +feature-based

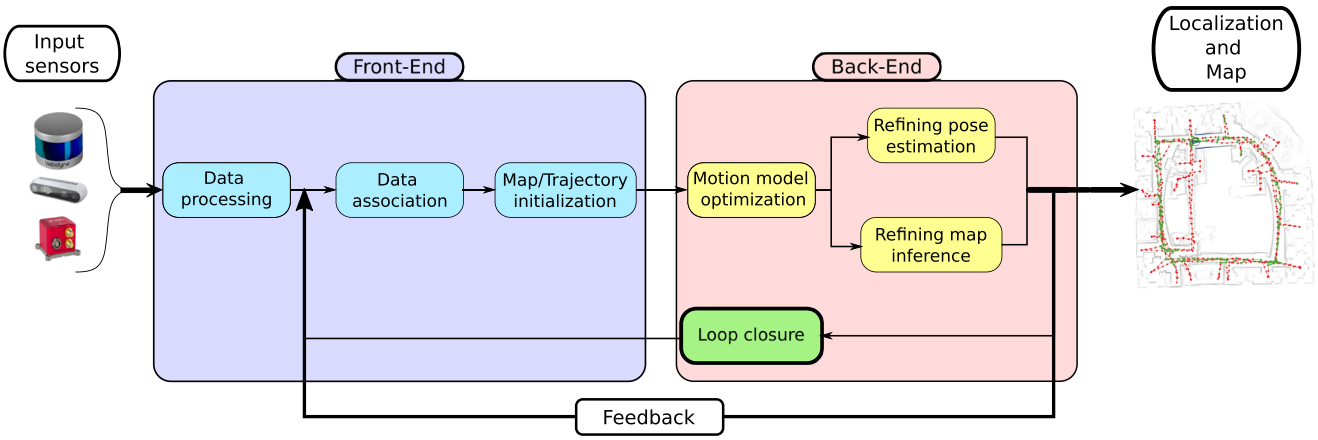


Fig. 7 Structure of a SLAM process

approaches the goal is to select areas in the constructed image from the acquired data that potentially disclose real-world qualities. Historically, features were defined in order to minimize the computational complexity generated by handling each pixel. Many feature extractors and descriptors have been developed, each with its own limitations and performances in terms of processing speed, rotation, scale and illumination invariance . Therefore, selecting a suitable feature detector depends on the targeted environment (aerial, terrestrial or aquatic) and the vehicle's computational power which ultimately creates a trade-off between computational speed and feature quality. More generally, features can be classified in three main categories; low-level features like geometric primitives (points, corners, edges, lines), middle-level features such as surface planes, or high-level features such as objects with semantic labels. In camera-based SLAM, there are several detectors and descriptors that can elicit and represent salient locations in an image, also known as keypoints. These features should be recognizable in the scene and unaffected by changes in viewpoint or illumination, in addition to being resilient to blur and noise. For instance, such features can be **ORB** (Oriented FAST and Rotated BRIEF) features as in **ORB-SLAM2** [86] or simply blobs and corners like **SOFT-SLAM** [22].

LiDAR Data Processing In case of 3D LiDAR, processing point cloud requires the use of one of two methods: inferring geometric information from 3D discrete and unsorted points or direct 3D point cloud registration. In the first case, geometric primitives can be categorized into shape primitives (lines, planes, cubes, etc.) or structure primitives (skeletons, 2D outlines, 3D edges, etc.). The geometric primitives (also called features) represent recognizable areas that are consistent in time and space. The extraction of these primitives can be performed using multiple methods such as segmentation, clustering, energy-based optimization, etc. Depending on the parameters tuning, these methods can be sensitive to noise and density or time consuming. For example, in [134], the authors of LOAM used a term to evaluate the smoothness of a local surface, allowing them to classify the points located on sharp edges and planar surface patches. In case of direct 3D point cloud registration, the main goal is to record these point clouds consecutively in order to align them in the same coordinates frame. This method can either combine incoming scans with portions of a pre-built point cloud map yielding to the localization of the vehicle or combine successive LiDAR scans in order to calculate the odometry of the vehicle. The strategy used in **IMLS-SLAM** [29], relies on removing dynamic objects and down-sampling the scan before optimizing the transformation using a scan-to-model strategy. Alternatively, in [78] the authors have designed a deep learning framework that takes as input the online LiDAR point cloud and a 3D point cloud map that is previously built and outputs centimeter-level pose estimation.

3.1.2 Data Association

After the identification of points that can be matched across multiple frames in feature-based approach or simply acquiring and matching the raw dense information in direct approach, the data association module establishes measurement correspondences across different frames. That is by associating each measurement to a specific landmark (3D point) in the environment. In other words, the set of measurements is linked to a subset of unknown variables. These correlations are then used as inputs to other modules. The most important condition for this module is the repeatability and the uniqueness of the key-points whether they are detectable features or simple pixels measurements. These conditions ensure successful tracking and increase certainty.

Direct Methods They are categorized into either dense or semi-dense methods. While dense methods utilize each and every pixel in a frame, semi-dense methods use only pixels with a brightness gradient exceeding a predetermined threshold. The fundamental concept of all direct methods is brightness consistency constraint which is based on the assumption that the corresponding pixel of a 3D point from a scene observed at an image acquired at time t_i will have the same intensity when observed in a future image acquired at time t_j. The matching problem is then solved by iteratively minimizing the squared pixel-intensity difference between the current image and the transformed preceding image. This non-linear optimization is a computationally heavy task that depends both on the number of the parameters in the considered transformation and the number of pixels in each image.

Feature-Based Methods Once the feature is selected, establishing correspondences for low-level features can be achieved by matching features in two images (2D-2D), by matching a 3D point in the world frame with its projection onto the image frame (3D-2D), or by matching 3D point in the world frame with the reconstructed map (3D-3D). In the first case (2D-2D matching), the association is carried out by searching within a window surrounding the feature's location in the second image. As a result of epipolar geometry, this search is reduced to one dimension (along a line), as long as the camera pose is successfully estimated (the transformation between the two images is well known). Given the type of the chosen feature descriptor, various quantities exist to evaluate their similitude, such as L1/L2 norms or Hamming distance. However, such measures can reduce the real-time performance of the system

because they are time-consuming. To tackle this issue, special approaches are introduced to accelerate the search such as KD-tree [83] or Bag-of-Words [38]. In the second case (3D-2D matching), the association is established by deducing the current camera pose from the previous poses then projecting the 3D landmark onto the corresponding image frame. The 2D projection is then matched to the 2D features using the before-mentioned 2D-2D matching technique. This type of matching is implemented when the camera pose has to be estimated from the structure of the environment. In the third case (3D-3D matching), the main purpose is to calculate and rectify the cumulative error along the path traveled. This technique where 3D landmarks of both ends of the loop are matched in order to estimate the transformation parameters between the two ends allows loop closure in the Back-End block, which leads ultimately to a drift-free trajectory. Figure 8 shows a comparison of trajectories with drift and without loop closure and Fig. 9 shows an illustration of matching techniques for low-level features. Associating middle-level features like planes or any image regions of homogeneous texture is accomplished by constraining the distance between the feature in the current frame and the previous ones in the constructed map to be less than a particular threshold. Otherwise, the feature is added as a new landmark to the map. The same technique is used to associate high-level features as described in the work of Bowman et al. in [15].

Deep Learning Methods The capabilities that Artificial Intelligence can provide to autonomous vehicles are well recognized and as a result it is the subject of topical research in recent years. It has been widely used due to its ability to continuously improve some classical algorithms performances. Indeed, robust, real-time detection of obstacles is a critical requirement from a safety standpoint. Traditional feature extraction algorithms that classical camera-based SLAM systems rely on have difficulty dealing with textureless regions and other complex scenes, which limits the development of camera-based SLAM. The studies of feature points extraction adopting deep learning show that this method has more advantages than traditional methods in dealing with complex scenes. Recently proposed neural networks have demonstrated great success also in estimating 6-DoF poses [90] or pose graph optimization [68]. When compared to hand-designed methods for the same tasks, these methods are strong on robustness, since they will always make predictions that are similar to real scenarios present in their training data. But designed methods still often have advantages in flexibility in a range of unforeseen scenarios, and in final accuracy due to the use of precise iterative optimization.

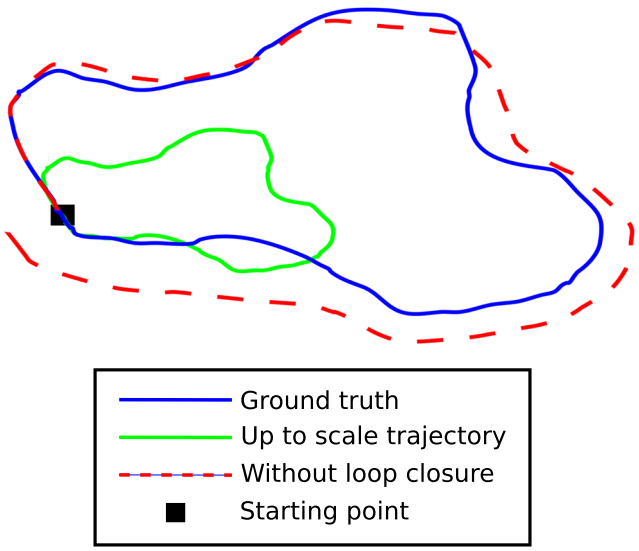


Fig. 8 Drift and loop closure impact on trajectory

3.1.3 Map/trajectory Initialization

Before starting to estimate accurately the pose and the map, a definition of the coordinate system and the unknown environment are required. It is then the final task of the Front-End Block to provide an initial guess for the estimated variables to be optimized in the next modules. Usually, this is accomplished by computing the spatial transformation between successive frames to calculate an initial set of map points. In **ORB-SLAM** [85] this step is executed by computing in parallel two geometric models: a homography assuming a planar scene and a fundamental matrix assuming a non-planar scene. The selected criteria to chose the appropriate model is the symmetric transfer error [4]. When the localization process loses track or when insufficient features are detected in subsequent frames the initialization process is resumed.

Other SLAM algorithms such as **LSD-SLAM** [35], **DSO** [34] or **DPPTAM** [21] use a random initialization process taking into account only one frame where pixels with high intensity gradients are given random depth values with an associated large variance. Consequently, many frames need to be processed before converging to a solution with minimum depth variance.

3.2 Back-End

This block's main task is to optimize the poses obtained to reduce the error caused by the geometrical constraints. It can also be described as the computational module responsible for the higher level processing of the acquired knowledge from the Front-End. This block corresponds to the core of SLAM as an estimation problem. A mathematical description of SLAM can be solved using three main approaches that we present in the following.

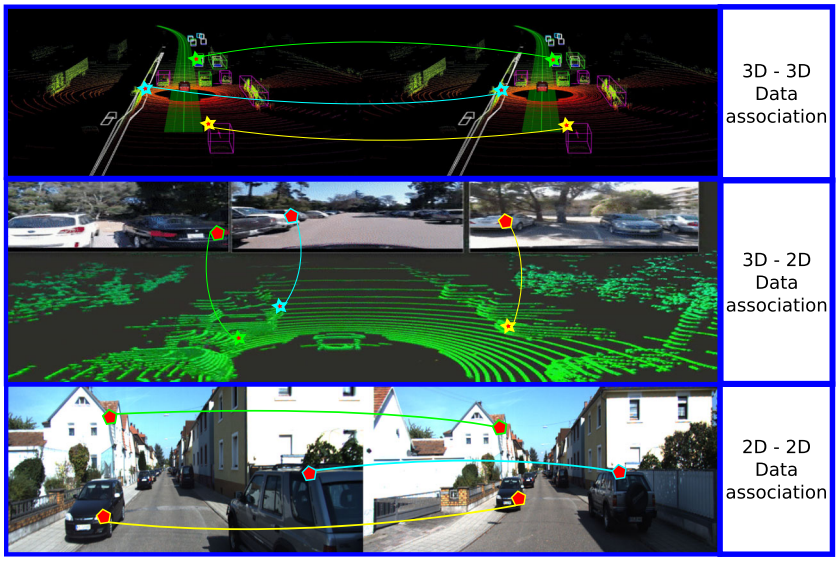


Fig. 9 Data association for low-level features

3.2.1 Filtering Methods

Filter-based methods derive from Bayesian filtering and work as two-step iterative processes that aims to compute the probability distribution given by Eq. 1 for all times k.

$$\mathcal{P}(x_k, m|Z_{0:k}, U_{0:k}, x_0) \tag{1}$$

Given that the Bayes Theorem is used to determine the joint posterior based on control u_k and observation z_k; A recursive method that starts with an estimate for the distribution $\mathcal{P}(x_{k-1}, m|Z_{0:k-1}, U_{0:k-1})$ at time $k-1$ can solve the Eq. 1. The state transition is considered under the assumption that it is a Markov process where the next state x_k is only affected by the previous state x_{k-1} and the exerted control u_k, and is unaffected by the observations or the map.

This prediction can also be expressed in terms of the state x_k and map m at a time k based on all observations $Z_{0:k}$ and all control inputs $U_{0:k}$.

Specifically, the recursive procedure for calculating the joint posterior (quantified by Eq. 1) is given by Eqs. 2 and 3. Where the recursion is determined by combining the **motion model** $\mathcal{P}(x_k|x_{k-1}, u_k)$ and the **observation model** $\mathcal{P}(z_k|x_k, m)$.

Prediction (Time-update)

$$\begin{aligned}&\mathcal{P}(x_k, m|Z_{0:k-1}, U_{0:k}, x_0)\\ &= \int \mathcal{P}(x_k|x_{k-1}, U_k)\\ &\times \mathcal{P}(x_{k-1}, m|Z_{0:k-1}, U_{0:k-1}, x_0)\, dx_{k-1}\end{aligned} \tag{2}$$

Correction (Measurement Update)

$$\begin{aligned}&\mathcal{P}(x_k, m|Z_{0:k}, U_{0:k}, x_0)\\ &= \frac{\mathcal{P}(z_k|x_k, m) \times \mathcal{P}(x_k, m|Z_{0:k-1}, U_{0:k-1}, x_0)}{\mathcal{P}(z_k|Z_{0:k-1}, U_{0:k})}\end{aligned} \tag{3}$$

The first branch of the filter-based methods concerns derivatives of the Kalman Filter (KF) such as Extended Kalman Filter (EKF) [96], Adaptive Extended Kalman Filter (AEKF) [121] or Information Filter (IF) [73]. This method decreases the uncertainties and provides an estimation at each stage of the process. However, it is extremely susceptible to erroneous data association. Moreover, updating all landmarks and associated covariance matrix on a regular basis necessitates a significant amount of computational power.

The second main branch in filtering SLAM algorithms is based on Particle Filters (PF). The theory behind is based on the use of a sample of particles to represent the state given its probability density. Then, as with every filter, a prediction of each particle's displacement is fulfilled and an update is conducted based on the observation. Particles are weighted according to their probability in relation to the measurements during the update phase. The most likely particles are retained, while the others are discarded and new ones are created [44]. Particle filter has become widely used and a large range of approaches based on it have excellent estimate precision and resilience.

Overall, filter-based SLAM are very dependent on their ability to successfully handling uncertainties and the probabilistic filter consistency. However, their convergence becomes slower in higher dimensional environments. This approach has deprecated in the last years favoring the emergence of graph-based and deep learning methods.

3.2.2 Pose Graph Optimization Methods

The majority of optimization-oriented SLAM techniques include two subsystems. The first one uses sensor data by finding matches between newly added observations and the

map in order to identify the problem's constraints. The second subsystem calculates or refines the vehicles's pose (and previous poses) as well as the map given the constraints to obtain a consistent whole.

Suppose we want to estimate an unknown variable χ that depends on the vehicle's trajectory and the position of the landmarks in the environment. Given a set of measurements $Z = \{z_k : k = 1, ..., m\}$ such that each measurement may be written as a function of χ, i.e., $z_k = h_k(\chi_k)+\varepsilon_k$, where $\chi_k \subseteq \chi$ is a subset of the variables, $h_k(.)$ is a known function (the measurement or the observation model) and ε_k represents a random noise. In Maximum A Posteriori (MAP) estimation, we estimate χ by computing the assignment of variables χ^* that attains the maximum of the posterior $p(\chi|Z)$; corresponding to the belief over χ, given the measurements:

$$\chi^* = \underset{\chi}{argmax}\ \ p(\chi|Z) = \underset{\chi}{argmax}\ \ p(Z|\chi)p(\chi) \tag{4}$$

The equality follows from the Bayes theorem. In Eq. 4, $p(Z|\chi)$ is the likelihood of the measurements Z given the assignment χ and $p(\chi)$ is a prior probability over χ. Assuming that the measurements are independent (the corresponding noises are uncorrelated), Eq. 4 factorizes into:

$$\begin{aligned}\chi^* &= \underset{\chi}{argmax}\ \ p(\chi)\prod_{k=1}^{m} p(z_k|\chi) \\ &= \underset{\chi}{argmax}\ \ p(\chi)\prod_{k=1}^{m} p(z_k|\chi_k)\end{aligned} \tag{5}$$

The problem described by Eq. 5 can be interpreted in terms of inference over a factor graph where the variables will correspond to the nodes. The terms $p(z_k|\chi_k)$ and the prior $p(\chi)$ are called *factors*, and they encode probabilistic constraints over a subset of nodes. A factor graph is a graphical model that encodes the dependence between the k-th factor with its measurement z_k and the corresponding variables χ_k.

Like filter-based methods, we can divide optimization methods into two main branches: Bundle Adjustment and graph-SLAM. These two branches stem from a MAP formulation. However, graph-SLAM includes a broad variety of sensor models and does not constrain the *factors* in Eq. 5 to model projective geometry as in Bundle Adjustment. Moreover, graph-SLAM needs to be solved incrementally by taking into account new measurements when they are made available at each step as the vehicle moves.

Fast linear solvers have been designed to update the estimate of χ as new observations are acquired (e.g., GTSAM [28], g2o [64], Ceres [2], iSAM [56], SLAM++ [94]). They can solve problems with tens of thousands of variables in a matter of a few secondes.

Bundle Adjustment is a technique that jointly optimizes the 3D structure and the pose. The majority of early research concentrated on 3D reconstruction but it has since then been applied to SLAM. The fundamental concept is to optimize an objective function commonly using the Levenberg-Marquardt algorithm [93]. The latter reduces the reprojection error (gap between observations in the image and reprojected previous features) resulting in the optimal vehicle and landmark positions. However, because it examines all of the variables at once to optimize over, Bundle Adjustment technique is computationally intensive.

Bayesian SLAM's graphical form is particularly well suited to a resolution using optimization approaches. A matrix defining the associations between landmarks and vehicle positions may be simply generated and used in an optimization framework based on this graphical representation.

The graphical representation of Bayesian SLAM is particularly well adapted to a resolution via optimization methods. Based on this graphical representation, a matrix describing the relationships between landmarks and vehicle poses can easily be built and used in an optimization framework.

Generally, pose graph based methods are highly coupled with the performance of the front-end. Additionally, they are more memory-efficient and conventionally more accurate than filtering methods [116]. Therefore, they are the state-of-the-art in current back-end SLAM systems. We encourage the reader to find a tutorial on graph based SLAM in [43].

4 Mono-modal SLAM Algorithms

After establishing the importance of using Camera and LiDAR in modern SLAM applications such as autonomous vehicles and the theoretical background of SLAM, this section presents a state-of-the-art of the mono-modal SLAM algorithms. That are frameworks that use either camera or LiDAR. In case of camera-based SLAM, the classification is carried out according to the methodological approach used and the sensor family. For LiDAR-based SLAM the classification is conducted following to the global strategy. At the of each mono-modal-based SLAM category a comparison and a discussion of open problems is presented.

4.1 Camera-based SLAM

4.1.1 Research Status

Historically, camera SLAM was the first developed SLAM algorithm and it has since been a very dynamic subject in terms of research and contributions. Moreover, existing

camera sensors provide large amounts of possibilities both in quantity and quality of relevant data to be processed. Indeed, they are usually less expensive than other sensors, all while being easier to use and more compact. Table 2 presents a summary of the most representative camera-based SLAM that can be applied in the autonomous ground vehicle context, showing the main techniques used in recent years. Previous works, such as [37, 119] presented detailed surveys on camera SLAM and summarized its technical categories as feature-based methods, direct methods and RGB-D camera-based approaches. However, recent technical achievements suggest adding event-camera [99] and deep learning based approaches to the classification [129].

Feature-Based Methods Usually seen as the classical technique, the key idea behind Feature-based methods is to extract and track point features in the surroundings of the vehicle, while using geometric reprojection constraints during estimation. This method can be executed using one of the two existing methods: using a filter or using Bundle Adjustment (BA) technique. In the first approach the past posed are marginalized and the obtained knowledge is represented with a probability distribution, this technique was first used by **MonoSLAM** [24, 25] using an Extended Kalman Filter. This technique's accuracy proved to be limited when used in large-scale environments due to the limited number of features in the constructed map. In order to address this limitation **PTAM** [60, 61] adopted the BA approach where only a sub-set of past frames are selected to process. This subset is called keyframes and contain a high number of features. For a more complete comparison of the two techniques, one can refer to [116]. **PTAM** was also the first algorithm to propose the idea of a parallel threads for tracking and mapping. **ORB-SLAM** [85] built on the success of this framework in small environments to propose a more stable algorithm using three threads for tracking, mapping and loop closing. Using ORB features and Bag-of-Words [38], this technique is able of efficiently preventing cumulative errors, it has been extended to stereo and RGB-D cameras in **ORB-SLAM2** [86]. ORB-SLAM3 [17] has recently increased the algorithm's robustness and accuracy by using inertial information. In **ENFT** [131] the authors proposed a SIFT-based novel method suggesting a non-consecutive feature tracking to match tracks over one or multiple sequences. Recent works like **TagSLAM** [92] and **UcoSLAM** [84] consider adding fiducial markers as artificial landmarks placed in the environment to achieve long-term robust tracking and relocalization.

Direct Methods This method doesn't need to extract features, yet solves the pose problem by using raw pixel intensities which results in including a higher percentage of the available information. In 2011 Newcombe et al. proposed **DTAM** [88] which is one of the first direct methods, it computes dense depth map by minimizing a global spatially regularized energy function. Engel et al. proposed **LSD-SLAM** [35] using regions of sufficiently large intensity gradient and Lie-algebra for depth reconstruction and pose tracking. This work has been expanded in 2017 to **DSO** [34], where a sliding window optimization algorithm is used to exclude old camera poses and points that leave the field of view of the camera. To ensure high accuracy and robustness of motion estimation, different from **LSD-SLAM**, this model includes a complete photometric calibration that takes into account non-linear response functions, lens vignetting as well as exposure duration. In **SVO** [36] a hybrid approach is proposed, combining features with a direct method to track some keypoints (such as corners), then, using the information around the keypoints, estimate the camera's velocity and location. This method is fast because it doesn't require to extract features in every frame, but adds special constraints and optimizations. Furthermore, like **DSO** it doesn't include loop closure detection. Unlike classical direct-methods based on photometric error minimization, **NID-SLAM** [91] used the Normalised Information Distance (NID). This metric is more resistant to changes in the scene's appearance caused by lighting, weather, and structural variations. Its tracking approach incorporates depth uncertainties into the estimated pose solution. Finally, long-term accuracy is achieved by using NID for depth map updates.

RGB-D Camera Methods KinectFusion [87] was the first method to use RGB-D camera to construct dense 3D depth maps for tracking and mapping. The algorithm used Truncated Signed Distance Function (TSDF) to determine the surface of the scene that can be viewed in the current perspective by performing a global fusion of all depth maps in the volume and ray projection technique. To cut down on the hefty computational cost, [102] proposed to unify coplanar points to decrease the amount of the data processed using a GPU. In the same year Henry et al. introduced RGB-D mapping [47] for indoor environments. The algorithm used SIFT features from color images and the corresponding depth information. Matching 3D feature points was performed using RANSAC method that acts as an initialization for Iterative Closest Point (ICP), which is a local optimizer to achieve a more accurate pose. Binding visual and depth information together, this method is able to provide view-based loop closure detection. More recent research focused on combining RGB-D camera with other sensors. Laidlow et al. proposed in [65] the first tightly-coupled dense RGB-D-inertial SLAM system. The algorithm minimizes a combined photometric, geometric and inertial function in the tracking

Table 2 Summary of the most representative Camera-based SLAM systems, in chronological order. RTE is the Relative Translational Error as used in [39]

References	Year	Method	Hardware architecture	Output rate	RTE	Loop closure	Back-end optimization	Map representation
D3VO [129]	2020	Deep learning-based	N / A	N / A	0.7%	✗	–	Metric map (Point cloud map)
SOFT-SLAM [22]	2018	Feature-based	PC with quad-core processor @2.4GHz	@20Hz	0.81%	✓	Pose-graph optimization	Octomap
ORB-SLAM2 [86]	2017	Feature-based	PC with quad-core processor @2.4GHz and 8GB RAM	Odometry @25Hz-30Hz and Mapping @5Hz	0.72%	✓	Bundle Adjustment	Metric map (Landmark-based map)
Stereo DSO [125]	2017	Direct	N / A	N / A	0.84%	✗	–	Metric map (Landmark-based map)
NID-SLAM [91]	2017	Direct	PC with quad-core processor and AMD Radeon R9 295X2 @10GHz and 8GB RAM GPU	Odometry @10Hz and Mapping @1Hz	0.7%	✓	Pose-graph optimization	Metric map (landmark-based map)
S-PTAM [93]	2015	Feature-based	PC with quad-core processor @2.20GHz and 8GB RAM	@18Hz	1.19%	✓	Pose-graph optimization	Metric map (Landmark-based map)

step to concurrently estimate the camera pose and velocity. In the mapping step, it constructs a fully dense 3D reconstruction of the environment. Globally, the RGB-D-inertial system outperforms the RGB-D-only version especially in aggressive motions and low photometric and geometric variations scenarios. Alternatively, to decrease the global drift caused by inertial measurements due to noise and biases, **KO-Fusion** [51] suggested using a tightly-coupled dense RGB-D SLAM system with kinematic and odometry measurements from a wheeled robot.

Event-Camera Methods SLAM is one of the typical scenarios where event cameras offer many advantages over other classical sensors. Their differential and asynchronous principle of operation suppresses temporal redundancy and emphasizes their ability to capture high-speed motions at a higher dynamic range (HDR). Therefore, event-cameras show great promise for challenging scenarios such as HDR feature tracking and/or rapid movements. However, being a novel bio-inspired sensor, the main challenge remains the development of new algorithms capable of processing unfamiliar stream of intensity changes, that are the events, which will unlock this camera's full potential. Recent works, like [58], have addressed this challenge, inspired by successful SLAM systems that used parallel threads. This method uses three filters operating simultaneously to jointly estimate the motion of the event-camera, the intensity gradients and the inverse depth of a keyframe. In order to design a more robust state estimation and propose an ultimate SLAM, [123] introduced the first hybrid pipeline in which standard frames, inertial measurements as well as events are all combined. When compared to using simply events and an IMU, this fusion showed a 130% increase in accuracy, and a 85% increase when using only conventional frames and an IMU. The quest for improving accuracy and efficiency continues with the exploration of stereo event-based SLAM [137]. In this method, mapping is achieved by optimizing an objective function intended to assess spatial and temporal consistency in stereo event streams, while tracking is performed by taking into account events and a local map to compute the pose of the stereo camera with respect to the map.

Deep Learning-Based Methods Nowadays, deep learning has become the mainstream technique used in computer vision and image recognition. By contrast to traditional methods, it can achieve higher accuracy and can be easily added to SLAM framework as external and/or inherent modules to recognize loop closure, to estimate inter-frame motion or even to create semantic maps of the environment. Building on ORB-SLAM2, **Sunderhauf et al**. [117] proposed a semantic mapping system that reconstructs a 3D model of the environment including both identified objects and non-identified object structures. Their system uses a Convolutional Network to detect and classify objects. Thus, every object detected is assigned to a segment of 3D points by an unsupervised 3D segmentation algorithm. In contrast, **Fusion++** [80], presented an online object-level SLAM system capable of using arbitrarily reconstructed objects to build a precise and consistent 3D graph map. They are then employed for tracking, loop closure detection and relocalization after being progressively corrected via depth fusion. However, the challenge remains to find a good balance between ensuring sufficient scene coverage and filtering detections. Apart from semantic SLAM, efforts have been made in [5] to tackle the problem of loop closure detection by proposing **NetVLAD**. Inspired by Vector of Locally Aggregated Descriptors (VLAD) used in image representation, the authors designed a new Convolutional Neural Network architecture that is trained for place recognition. As this technology advances, modern research uses deep learning to govern the whole process of SLAM. The work of Liu et al. in [71] improved the performance of landmark-based methods for place recognition using CNNs. This approach suggested using feature maps from the low layer of a CNN to extract landmarks and compact landmark descriptors from high layers to perform quick matching. Finally, in the refinement step, Markov random field (MRF), combined with Bayes filters, are exploited to perform robust matchings. Recently presented **DeepSLAM** [67] suggests using an unsupervised learning framework that implements several essential components, including Mapping-Net, Tracking-Net, Loop-Net and a graph optimization unit. Similarly, **SimVODIS** [59] is built on top of Mask-RCNN [46] which allows the design of networks that take advantage of rich features to estimate the pose, predict the depth map and detect objects. The system uses a single thread to complete all these tasks and manages to outperform or match the performance of the other state-of-the-art methods.

4.1.2 Open Problems

Although camera-based SLAM has been very well studied in recent decades and has achieved a lot of success, there are still many problems that remain unsolved due to hardware and/or software limitations. On one hand, even if approaches that use deep learning are successful, they are still limited by untrained cases and their learning behavior remains unpredictable. On the other hand, feature-based classical approaches are very texture and light-dependent, while direct approaches require more photometric calibration and more computational power. Moreover, RGB-D methods are very range-limited and event-camera

techniques suffer from artifacts caused by event noise and low spatial resolution.

Robustness Achieving a more accurate camera pose estimation is conditioned by the consideration of the shutter type. While, most camera-based SLAM frameworks assume a global shutter, most cameras use rolling shutter due to its lower cost. Consequently, camera pose estimation is obtained by an interpolation approach which limits the global accuracy. Moreover, these solutions are still prone to errors due to their multiple sensitivities like illumination changing conditions, high dynamic environments, fast motions, strong rotations and low-texture environments. Even if each camera-based SLAM technique can be more resilient to some conditions it remains fragile under other circumstances [16]. The question remains, how can we build the ideal SLAM system capable to adapt to various environments or at least decrease the amount of input parameters that must be tuned for the SLAM to operate in a given scenario?

Flexibility The majority of proposed frameworks are very sensor and platform-dependent, causing the system to fail in case of changing the defined architecture. Such situations require using a modular and extensive set of sensors for fusion or opting for a learning-based system that can extract higher semantic information independently of the sensor. In this hypothesis, the challenge becomes how can we represent a unified map for the vehicle. One possible solution is to implement a cross-modal symbolism of the state and the environment, based on the employed sensing methods, that can efficiently manage data matching and storage [82]. Moreover, designing a system than can adapt the representation of the environment depending on the task, the complexity or the richness of the environment can solve problems such as memory management [50]. The creation of an optimal representation automatically will certainly facilitate long-term localization and mapping.

Scalability Modern SLAM systems still struggle with large scale areas where the autonomous vehicle should operate for an extended duration and through a long and versatile trajectory. In fact, the memory needed to store previously discovered landmarks will grow exponentially. Even if we suppose GNSS localization to be always reliable, either outdoor or indoor, the transition from one paradigm to the other could still cause drift or additional noise. For long-term autonomy, more techniques need to be investigated in order to deal with time-varying maps. This problem is accentuated in severely resource-constrained platforms. More attempts should be taken to suggest robust strategies that determine whether an information should be discarded, updated or permanently remembered [16].

Hardware To a certain extent, SLAM problem often is presented as a trade-off between responsiveness in real-time and accuracy. Direct methods and some feature-based methods often take advantage of GPUs and FPGAs to speed-up and design low size and weight embedded SLAM systems. However, efforts still need to be made to reduce the computational cost, capitalizing on the recent advances in these processors technologies. An approach that can integrate at design level strengths and weaknesses of the targeted platform will hopefully be an effective strategy to make use of domain specific processor [1]. On another note, developing more intelligent sensors that can bring the processing as close as possible to the sensing devices may also be an alternative solution to achieve hardware efficiency and concentrate efforts on the development of more reliable algorithms.

4.2 LiDAR SLAM

4.2.1 Research Status

Compared to camera SLAM, LiDAR SLAM is second in the amount of information in need of processing (See Table 1). It has become more and more common in many applications such as autonomous driving cars and unmanned aerial vehicles, mainly due to the quality of data they provide and the constant price drop across recent years. LiDAR SLAM can provide high-frequency range measurements with relatively small errors, it can track distances, capture full 360° 3D scans, extract key points belonging to edges and planes and generate point clouds of great size. Multiple approaches have been suggested to perform motion estimation and mapping using LiDAR data. Table 3 presents a summary of the most representative LiDAR SLAM that can be applied in the autonomous ground vehicle context, showing the main techniques used in recent years. With algorithms main focus being efficiency in their processing pipeline, LiDAR SLAM can be classified according to the granularity of the point cloud data they use for estimation or the technical approach. In particular, the main categories are:

Registration-Based Methods By analogy with camera SLAM, these methods can be seen as "direct" methods, because they make use of all the the points in the LiDAR data. These techniques take advantage of recent developments in accurate point clouds registration algorithms [120], in order to yield the transformation between two successive scans by aligning them in the same coordinates frame. This approach, also known as, scan matching is usually achieved by the Iterative Closest Point (ICP) algorithm [11]. ICP divides the registration in two sub-problems: the establishment of correspondences and the optimization of an error

Table 3 Summary of the most representative LiDAR-based SLAM systems, in chronological order. RTE is the Relative Translational Error as used in [39]

References	Year	Method	Hardware architecture	Output rate	RTE	Loop closure	Back-end optimization	Map representation
SuMa++ [20]	2019	Registration-based	Workstation with 8-cores processor @3.6GHz, 16GB RAM and Nvidia Quadro P4000 with 8GB RAM GPU	@10Hz	0.55%	✓	Pose-graph optimization	Semantic and metric map (Surfel map)
LO-Net [66]	2019	Deep learning-based	PC with quad-core processor @3.4GHz and Nvidia 1080 Ti GPU with 4GB RAM	@12Hz	0.83%	✗	–	Metric map (Point cloud map)
LeGO-LOAM [108]	2018	Feature-based	PC with quad-core processor @2.5GHz	Odometry @30Hz and Mapping @10Hz	0.55%	✓	Pose-graph optimiza-tion	Metric map (Point cloud map)
			Nvidia Jetson TX2 with 4GB RAM and quad-core ARM Cortex-A57 processor	Odometry @15Hz and Mapping @3Hz				
SegMap [32]	2018	Deep learning-based	PC with quad-core processor @4.0GHz and Nvidia GeForce GTX 980 Ti GPU	@30Hz		✓	Pose-graph optimization	Semantic map
SuMa [8]	2018	Registration-based	PC with quad-core processor @3.4GHz, 16GB RAM and Nvidia GeForce GTX 960 with 4GB RAM GPU	@20Hz	0.7%	✓	Pose-graph optimization	Metric map (Surfel map)
IMLS-SLAM [29]	2018	Registration-based	Single-core processor @4.0GHz and 1GB RAM	N / A	0.55%	✗	–	Metric map (Voxel map)
Google Cartographer [48]	2016	Feature-based	Workstation with 6-cores processor @3.2GHz and 2.2GB RAM	N / A	0.7%	✓	Pose-graph optimization	Metric map (Grid map)
LOAM [134]	2014	Feature-based	PC with quad-core processor @2.5GHz and 6GB RAM	Odometry @10Hz and Mapping @1Hz	0.88%	✗	–	Metric map (Point cloud map)

metric between all the points of both point clouds. The two procedures proceed iteratively until the convergence criteria is satisfied. Derived algorithms have been proposed with the aim to enhance ICP robustness, by using point-to-plane ICP [75] for instance or point-to-line ICP [19]. An extensive approach was proposed in Generalized ICP [105] with a probabilistic framework merging ICP and point-to-plane ICP. With the purpose of reducing inherent drift caused by consecutive registrations, methods like [81] suggested to combine a pose graph process with a loop closure mechanism using ICP algorithm for the localization. The pose graph is built by taking into account reference frames called keyframes. Even if ICP can be reasonably reliable, the point-wise matching is very sensitive to outliers and partially missing structures as it is only effective when the consecutive scans have sufficient overlap. Moreover, initial misalignment decreases the global performance remarkably.

In an attempt to tackle these issues, Normal Distribution Transform (NDT) [12] algorithm was proposed for 2D scans before its generalization to the 3D space in [79]. NDT's novelty is that it subdivides the space into cells before establishing grid-wise correspondences, unlike ICP where it is explicit and point-wise. First, each cell is assigned a normal distribution that models the probability of measuring a point, then Newton's algorithm is used to find the spatial transformation between the consecutive scans, resulting in a more robust algorithm. Other optimization algorithms can be used to solve this transformation problem such as Levenberg-Marquardt Method that combines Gradient Descent and Gauss-Newton Method. It should be noted that NDT is sensitive to the cell size. That is, if the size is too large, the precision decreases. Otherwise, if the size is too small, larger memory is required to store data. In practice, cells size is fixed by means of k-means clustering [23] or an Octree structure [33]. Finally, even if NDT needs overlapping percentage between the two scans like ICP, the additional required step of interpolation between cells increases the computational cost.

One of the numerous SLAM algorithms that relies on a scan-matching framework is Surfel-based Mapping (**SuMa**) [8]. In this method, a point cloud is converted into a vertex map and normal map while simultaneously creating a 3D representation of the scan using surfel (Surface elements) map. The next step corresponds to using the vertex map and the normal map to estimate the odometry by using projective data association (denoted frame-to-model ICP in this context). The current pose is employed to update the surfel map by adding unseen areas and refining covered ones. Afterwards, the current surfel map is used to detect whether a place has been revisited. Parallel to this, a pose graph composed of loop closure information and relative poses odometry is optimized. The surfel map is finally updated using the resulting optimized poses. This strategy was extended in **SuMa++** [20] taking advantage of several semantic classes. The point-wise labels are then obtained by spherical projections of the point cloud. This additional information enables the algorithm to filter outliers caused by dynamic objects and add more semantic constraints in the data processing step, which results in a more accurate and robust pose estimate.

A different method based on scan-matching approach is Implicit Moving Least Squares SLAM (**IMLS-SLAM**) [29]. Its first step is to eliminate dynamic objects before the scan is matched to the model map. This algorithm, simplifies this task by discarding small objects (assuming they could be dynamic objects) and the ground points. Then, a down-sampling strategy is applied to select only the points with more observability. Finally, a scan-to-model matching strategy is performed to optimize the transformation using IMLS surface representation. Without loop closure detection, the proposed framework performs at the same level of state-of-the-art algorithms.

Feature-Based Methods Initially inspired by features used in 2D context, these methods use suitable 3D features to estimate motion between scans. Therefore, using only a subset of LiDAR data that corresponds to regions of interest. Descriptors are then used to establish correspondences between the detected features of consecutive point clouds. Many 3D features have been proposed in literature, for instance Guo et al. [45] proposed Rotational Projection Statistics (RoPS) feature descriptors, Fast point feature histograms (FPFH) were suggested in [101] and Normal Aligned Radial Feature (NARF) were presented in [115].

LOAM [134] is a feature-based robust LiDAR odometry and mapping algorithm that can achieve highly accurate results in real-time. The algorithm runs two threads in parallel; one for odometry at a high frequency but low fidelity to estimate velocity of the LiDAR and the other for mapping at a relatively lower frequency. The first thread starts by extracting points that are on sharp edges and planar surfaces provided they are not parallel to laser beams or in occluded regions. These features are then matched witch patches of points from the subsequent scan. Finally, the motion estimation is solved using Levenberg-Marquardt method. The second thread is run in the background and is used for the registration of the point cloud. Combination of the two threads allows this method to achieve reasonably satisfying results this method. However, it still suffers from motion estimation drift caused by the absence of loop closure detection. In order to tackle this issue, **LeGO-LOAM** [108] proposed an extension of this method that can improve speed and reach more accurate results by adding loop closure and global optimization. Optionally, the system can take as input IMU data. The major contribution of this framework is making use of the ground segmentation

to remove unreliable features, then using a two-step Levenberg-Marquardt optimization first for planar features then for edge features to speed-up the process. In recent work **SC-LeGO-LOAM**[1] that combines using **LeGO-LOAM** [108] and *Scan Context* [57] as spatial descriptor that uses structural information for place recognition and global localization. The presented pipeline is light-weight and achieves reasonable time performance by using KD-tree to search for the *Scan Context* while running loop detection at 10-15Hz.

Google Cartographer [48] is another LiDAR SLAM that provides long term loop closure. Although the original algorithm was created for 2D LiDAR, it has been generalized to 3D. In this method, few consecutive scans are used to create a submap that is shaped as a probability grid. All the points in the grid and all the points that are nearest to each one of them are used to define the matching pixel. Their probabilities are then adjusted, depending on the scan's hits and misses whenever the submap is updated with the insertion of a new scan. A Ceres-based [2] scan matcher is used to optimize the scan pose relative to the current submap before effectively inserting the scan into the submap. In parallel, using Sparse Pose Adjustment [63] all the poses of all scans and submaps are optimized. Loop closure optimization, like scan matching, is also solved using Ceres. The search is accelerated using a branch-and-bound approach. These techniques have enabled the authors through their algorithm to deliver real-time fully optimized maps that cover an area of tens-of-thousands of square meters.

Alternately, **CPFG-SLAM** [54] focused on off-road environment and suggested using surface and line features to describe the characteristics of point cloud in a probability grid. This algorithm uses ICP and NDT as reference and computes the nearest neighbor grid instead of nearest neighbor point to reduce feature extraction and search cost. Lastly, Expectation-maximization (EM) is used to provide a pose estimation iteratively, while the optimization problem is solved by the Levenberg-Marquardt algorithm. Even if this algorithm is more accurate and reliable in challenging environments compared to others in the state-of-the-art, it is still not adapted to urban environments and its performance is drastically affected by dynamic objects.

Deep Learning-Based Methods New algorithms have emerged that use deep learning techniques using LiDAR data showing promising results in feature detection, semantic segmentation or localization. These methods either try to solve the SLAM problem in an end-to-end fashion by using the raw point clouds as inputs and directly estimating the vehicle's motion, or by trying to substitute certain modules of an existing classical SLAM framework that could take advantage from the achievements of a deep learning networks.

For instance, **PointNetVLAD** [122] solves large scale place recognition through the extraction of local features before clustering them into **VLAD** [6] global descriptor. **SegMap** [32] proposed using the 3D point clouds to extract 3D segments to solve the localization and mapping problem by using a map representation solution. These 3D segment descriptors are extracted using a Convolutional Neural Network with an architecture that was developed by varying several parameters before settling for an ultimate configuration where two fully connected layers are employed with three 3D convolutional layers placed around max pool layers. Localization is then achieved by identifying correspondences using k-Nearest Neighbors (k-NN). Finally, the same features are used for reconstructing the environment and for extracting semantic information. This approach, addresses the computationally intensive task of processing 3D points clouds by using segments as compressed data. Although this method achieves good accuracy, it still has limited applications in real-time tasks. Inspired by RGB semantic segmentation methods, **PointSeg** [126] tackles this issue in order to balance efficiency and real-time performance. Firstly, LiDAR points cloud is transformed by projection into a spherical image then features are extracted from the dense generated data. Secondly, a point-wise semantic mask is predicted using an improved version of Squeeze-Net [53].

The common obstacle that the majority of the previously mentioned strategies face is the presence of dynamic objects (such as cars, bicycles, pedestrians, etc.) in the scene which limits the global performance of the system. One approach adopted by many researchers to overcome this challenge is to detect and delete these dynamic objects. **LO-Net** [66] presented a novel encoder-decoder network architecture that estimates odometry using consecutive scans while concurrently learning to mask dynamic regions and to estimate the surface normal. Moreover, this method better formalizes the odometry learning process while providing higher-order interaction between successive scans by incorporating a spatio-temporal geometry consistency constraint to the network. Furthermore, it designs a scan-to-map module, which improves the estimate's accuracy by using the semantic and geometric information learned. However, this method is still limited by the dataset used for training which restricts its application scenarios, plus it could be more practical to use direct 3D point clouds instead of the the encoded data.

A novel learning-based LiDAR localization system capable of achieving centimeter-level accuracy was demonstrated in $\boldsymbol{L^3}$-Net [78]. The input of this framework is formed by a set of LiDAR points with their corresponding

[1] Available from https://github.com/irapkaist/SC-LeGO-LOAM

global coordinates and a predicted pose that is often generated by an IMU, which constitutes a map built offline, plus an online LiDAR scan. Then, local feature descriptors are extracted from a set of keypoints using a mini-version of PointNet [97]. Afterwards, a cost volume is constructed and regularized with 3D Convolutional Neural Networks (3D CNN). Later, a Recurrent Neural Network (RNN) is introduced to achieve temporal smoothness of the displacements predictions. This strategy has been extended by the same authors in **DeepICP** [76] and **DeepVCP** [77] where they were inspired by Iterative Closest Point (ICP) and Virtual Corresponding Points (VCP) and tested to different Neural Networks architectures to estimate transformation and find the target point cloud's matching points. The main originality is adding a weighting layer that uses a learning process to assign each extracted feature a corresponding weight. Moreover, to obtain higher accuracy in the learning-based registration task, a loss function that incorporates both the local similarity and the global geometric constraints is included.

4.2.2 Open Problems

Even if LiDAR SLAM is proven to provide quite satisfying results, the employed technology still faces some obstacles. Indeed, the sensor position and the properties of the surrounding objects can easily affect the precision of the acquired measurements. Moreover, information is lost if objects are of inconsistent shape or when occlusions occur due to sudden scene changing. Furthermore, more computationally efficient registration algorithms need to be developed in order to deal with the vast amount of the acquired point clouds. While registration based methods are considered more computationally greedy, feature based approaches can generate accurate results in a reasonable time. However, depending on the environment, multiple parameters need to be tuned. Deep Learning based methods start to gain notoriety in LiDAR SLAM context but multiple considerations should be taken into account such as time of inference and the representability of the training dataset, plus the numerous architectures that should be tested before validation which sometimes seems to be more of an *ad-hoc* solutions.

Scale and Texture of the Environment Usually the angular resolution of LiDAR in the vertical direction is lower than in the horizontal direction. This have drastic impacts on the overall performance of LiDAR SLAM when the autonomous vehicle is surrounded by tall repetitive structures or low-textured environments [18]. Additionally, it is affected by large-scale environments such as large fields. Consequently, the point clouds become very sparse or the distance exceeds the maximum range [54]. Moreover, operating for an extended period of time and over long trajectories will require additional memory for the system to remember previously visited areas.

Dynamic Scenarios In dynamic real-time scenarios, LiDAR transmits data in units of packets that form a complete field-of-view. while constantly moving. Therefore, at each frame the point cloud is not at the same coordinate system and is not equal to a point cloud of the same frame in case of a static scanning. This generates motion distortion and yields to erroneous results. Although, some algorithms try to correct the point cloud distortion, motion compensation con not be guaranteed [108]. Furthermore, accurately removing dynamic objects from a scene is still an open problem. In fact, filtering moving objects from the scene can be challenging, especially when the autonomous vehicle is running at an important speed. Moreover, filtering-out all moving objects can limit the estimation of the pose or velocity and will surely be at an additional computational cost [20]. Other approaches try to solve this problem by maintaining the map of some location over a period of time by adding missed features or removing transient map components [10].

Adversarial Sensor Attack The work of Shin et al. [111] presented a spoofing by relaying attack, which can make the illusions look closer than the position of the spoofing device as well as induce spurious responses in the LiDAR output. The experiments conducted showed that the attacker can deceive the targeted sensor by simulating a fictional circumstance via a created attacking signal. When exposed to these simulated impulses the sensor perception is unable to differentiate between a fake and a real circumstance. Thus, by exploiting this semantic gap between the real environment and the perceived sensor's measurements introduced saturation attacks can entirely disable a LiDAR's ability to detect a certain direction. Therefore, such threats need to be further investigated in order to propose more resilient systems.

5 Multi-Modal SLAM Algorithms: Camera-LiDAR Coupling Strategies

Previous sections detailed advantages and disadvantages of using camera-based or LiDAR-based SLAM. On one hand and depending on the chosen sensor, to an extent, camera SLAM can still provide satisfying results. However, in case of monocular cameras, it is still limited by the scale drift or the poor depth estimation. RGB-D and stereo-cameras face ranging constraints in case of outdoor scenarios. Plus, event-cameras have spatial resolution cap that restrains their use. On the other hand, LiDAR SLAM is proven to be efficient

facing these challenges, even if it usually need greater computing power. Alternatively, using both techniques in a multi-modal framework demonstrated great potential to solving many issues in the field. Specifically, using multiple sensing modalities is a necessity when the aim of SLAM is to ensure maximum data integrity in case of system failure. Moreover, the two mainly used sensing modalities in literature are complementary in the sense of each surpasses the limitation of the other (Table 4).

5.1 Loosely-Coupled Strategies

Loosely-Coupled approaches treat camera-based SLAM and LiDAR SLAM as two separate modules running at different rates and exchanging information in a statistically optimal way. Therefore, the system is designed to fuse multiple sensor estimates by updating them at the correct time interval. Which ultimately, smooths the motion and the map based on the provided measurements and uncertainties. It is relatively easy to implement but usually suffers from accuracy problems.

One of the first attempts to take advantage of both camera and LiDAR data was carried out in [89] that used visual appearance signatures for loop detection in LiDAR-based SLAM. While laser measurements are used to build a 3D map of the environment progressively, loop closure detections are carried out using a similarity metric computed over consecutives images instead of a single image. Further, a technique based on eigenvalues, that considers only the atypical or interesting matches, is used on the similarity matrix to decrease the false positives ratio.

In 2014, Zhang et al. presented **DEMO** [132, 133] that used depth information from LiDAR to recover camera motion. Harris corners features are extracted and tracked from RGB images, if the corresponding depth information is available from laser data it is directly associated to it, otherwise it is triangulated using the previously approximated motion. The core of the method, is a Bundle Adjustment that refines the motion estimation concurrently by optimizing sequences of images. Although this method provides accurate results, it is still very dependent on tracking features that are not always available in homogeneous environments. Moreover, it does not take full advantage of LiDAR data and uses only a fraction of the measurements to assign their depth to 2D camera features. This strategy, does not only exclude most of dense LiDAR points from use, but also increases residuals errors accumulated by the flawed depth estimation by triangulation. Moreover, it does not provide loop closure detection capabilities. This issue was later addressed by [69] in **VL-SLAM** using visual information provided by ORB features and the Bag-of-Words model to detect whether the vehicle has already visited the place. Scan matching is performed using Local Registration and Global Correlation (LRGC) where correspondences are made between point cloud and line segments. The Sparse Pose Adjustment (SPA) based on Levenberg-Marquardt algorithm is used for pose graph-based optimization to correct the global map. The proposed method is efficient in loop detection but using ORB features makes it vulnerable to featureless environments.

Lopez et al. presented in **MS-SLAM** [74] a framework that adapts to the sensory configuration of aerial vehicles, by combining several cutting-edge SLAM methods based on vision, laser and/or inertial measurements. The software implementation of the proposed SLAM system is based on three blocks corresponding to available open source packages. For camera SLAM, either ORB-SLAM or LSD-SLAM was used to estimate a 6-DoF pose. For LiDAR SLAM, HectorSLAM [62] was used to generate a 2D map. Scale correction is accomplished by adding other information from IMU and Extended Kalman Filter (EKF) is employed to fuse all available data, resulting in an improved pose estimation and a generated 2.5D local map. Unfortunately, a global consistent map can not be generated using this system as depth maps and point clouds are discarded once a new keyframe is obtained. Moreover, it can not provide loop closure detection.

VISO2-LOAM [128] is another loosely coupled Visual-LiDAR Odometry that combines the 6-DoF motion estimation output provided by **LibVISO2** [40] and **LOAM**. Stereo camera was used by the VISO2 module to calculate the transformation between two frames to correct laser point clouds. This motion estimate, combined with the last pose calculated by LOAM is used to initialize the current pose of LiDAR. Then, to match the point cloud collected in the map, shape characteristics from the corrected point cloud are extracted. The matching distance is reduced using the Levenberg-Marquardt technique to produce the ideal state of the current frame relative to the starting time. This strategy faces several challenges due to drift accumulated through distance and in environments lacking colors and textures.

LIMO [42] used a frame-to-frame motion estimation from viso2 features then leveraged LiDAR data to estimate the depth corresponding to those features. First LiDAR points are projected onto the image and a subset of points around a given feature and within a fixed size rectangle are chosen in order to extract local planes. To estimate depth associated to features lying on corners and edges, the algorithm uses histograms of depths with fixed bin width. Frame-to-frame odometry estimation is carried out using the Perspective-n-Point-Problem [118]. Moreover, landmarks are classified into near, middle and far points and are evenly distributed and employed to increase efficiency and robustness. Furthermore, semantic information is used to remove dynamic objects and determine the weight

Table 4 Comparison of coupled Camera-LiDAR SLAM systems, in chronological order

	References	Year	Method	Fusion level	Hardware Architecture	Output rate	Map Scale / Density	Dynamic	Map representation	Back-end optimization	RTE
Tightly-Coupled	**R²LIVE** [70]	2021	Features	Odometry estimation	Onboard computer with quad-core processor @1.80GHz and 8GB RAM	@10Hz	Medium / Dense	Low	Point cloud	–	0.21%
					PC with quad-core processor @3.6GHz and 32GB RAM	@25Hz					
	LIC-Fusion 2.0 [138, 139]	2020	Features	Data association	PC with quad-core processor @4.0GHz	C @20Hz L @10Hz	Small/Semi-Dense	Low	Landmark-based	–	0.17%
	TVLO [106]	2019	Features	Odometry estimation and Mapping	N/A	N/A	Large / Sparse	Low	Grid and Landmark-based	Pose-graph optimization	0.74%
	LC-LVF [55]	2019	Features	Odometry estimation	PC with quad-core processor @2.4GHz and 8GB RAM.	@7Hz	Small / Semi-Dense	Low	Grid and Landmark-based	Pose-graph optimization	0.25%
	VIL-SLAM [110]	2019	Features	Mapping	PC with quad-core processor @3.6GHz	N/A	Medium / Dense	Low	Grid	Pose-graph optimization	0.5%
	VLO-IMU [7]	2016	Features	Odometry estimation	N/A	N/A	Large / Sparse	Low	Landmark-based	Bundle-Adjustment	0.8%

Table 4 (continued)

	References	Year	Method	Fusion level	Hardware Architecture	Output rate	Map Scale / Density	Dynamic	Map representation	Back-end optimization	RTE
Loosely-Coupled	**LIV-LAM** [98]	2020	Features	Back-end optimization	PC with quad-core processor @2.4GHz and 8GB RAM	N/A	Small / Dense	Low	Point cloud	Pose-graph optimization	1%
	DVL-SLAM [112, 113]	2020	Direct	Data association	PC with quad-core processor @3.8GHz and 8GB RAM	C @20Hz L @10Hz	Medium / Sparse	Low	Point cloud	Pose-graph optimization	0.98%
	Pilatus Driverless [3]	2020	Deep learning	Data association	PC with quad-core processor @2.8GHz and 16GB RAM	@10Hz	Medium / Sparse	Low	Landmark-based	Pose-graph optimization	0.15%
	LIMO [42]	2018	Features	Data association	PC with quad-core processor @3.5GHz	@5Hz	Large / Sparse	High	Landmark-based	–	1.22%
	V-LOAM [132, 133, 135]	2017	Features	Data association	PC with quad-core processor @2.5GHz and 6GB RAM.	C @60Hz L @1Hz	Large - Small / Dense	High	Point cloud	–	1.16%
	MS-SLAM [74]	2017	Features	Odometry estimation	Distributed computing with onboard dual-core controller @1GHz and a PC with quad-core processor @2.4GHz	@25Hz	Small / Semi-Dense	Low	Landmark-based	Pose-graph optimization	1.25%
	VISO2-LOAM [128]	2017	Features	Odometry estimation	PC with quad-core processor @2.4GHz and 8GB RAM	@10Hz	Large / Dense	High	Point cloud	-	1.37%
	VL-SLAM [69]	2015	Features	Data association	PC with dual-core processor @3.3GHz and 4GB RAM	@5Hz	Small / Dense	Low	Grid	Pose-graph optimization	1.5%

RTE is the Relative Translational Error as used in [39]

of landmarks. Finally, Bundle Adjustment (BA) is used on the selected keyframes to estimate the ego motion. Even if this strategy shows good performance, it doesn't solve loop closure problem and using only a fraction of LiDAR measurements make accumulative errors increase significantly the longer the traveled path. Similarly, **DVL-SLAM** [112, 113] used the depth from LiDAR in a direct camera-based SLAM method, where photometric errors were minimized in an iterative way. This direct method is more advantageous in case of low resolution camera and sparse LiDAR measurements (reduced number of laser beams). Tracking is performed using a sliding-windows optimization that processes image frame and depth measurements. When performing pose-graph optimization, the pose of the oldest frame is marginalized. A place recognition-based module is added to perform loop closure. Finally, pose-graph optimization includes loop constraints and use g^2o [64].

LIV-LAM[98] is recently presented framework for SLAM that uses LiDAR-based odometry combined with monocular camera-based object detection and association in a pose-graph optimization fashion with loop closure detection. An unsupervised feature learning is performed on the fly to at three levels: low-level features, temporally-salient objects, and for discovered class correspondences, all without prior knowledge of the context. Every n frames a new keyframe is selected from which L2 normalized SIFT and L2 normalized color are extracted and fed into stream clustering to progressively learn a feature codebook (similar to a Bag-of-Features paradigm). Each tracked is associated with a Tracker Confidence Curve that can indicate whether a new object is discovered and should be integrated and is used for object correspondence inferring. Following the same methodology of **LOAM** and **LeGO-LOAM,** the authors use planar and edge features computed from LiDAR points to perform odometry. Next, these features are aligned and combined with the learned features via camera-based odometry, then added to the point-cloud map to find position transformation. Finally, the pose-graph optimization process is carried out using Levenberg-Marquardt algorithm. This method can still be improved to deal with outdoor scenarios and featureless areas.

One of the most recent research is presented in **Pilatus Driverless** [3] that introduces the perception, mapping and planning pipeline implemented on an autonomous race car. The perception setup is designed to detect track boundaries over a long range; consequently, two monocular cameras with global shutter in addition to two 32 channel rotating LiDARs are used. The longitudinal, lateral, and rotational velocities of the car are estimated by fusing raw data from all the input sensors, including an optical speed sensor, dual antenna GNSS, an IMU and motor resolvers with an Unscented Kalman Filter. The Vision pipeline is based on a Convolutional Neural Network, Tiny YOLOv3 [100], that is trained to detect the cones that limit the track. The distance to the boundary is then estimated and the detected cone is projected into 3D space. The LiDAR pipeline uses another Neural Network to predict the most likely color based on the LiDAR intensity pattern. Both pipelines can run autonomously which increases the system robustness. Point clouds that correspond to the cones are then projected onto both image planes to classify the far situated cones based on their color. Cone positions and uncertainties are filtered using a Kalman Filter, then a local map containing all cones within the new set observations is generated. Finally, a global map is produced in real-time using a GraphSLAM algorithm [43] with cones as the landmarks. Ceres-solver [2] is used to estimate the vehicle and cone poses together using non-linear least squares. Experiments show that this pipeline makes it possible to reach a maximum driving speed of 12 m/s in unknown environments. The corresponding map obtained has a RMSE of 0.29m. However, this solution is still not adaptable to real-life use due to its cost but mostly due to the complexity of scenarios where more agents and factors need to be taken into account.

5.2 Tightly-Coupled Strategies

Tightly-Coupled approaches combine the two SLAMs into one optimal framework. Most tightly-coupled optimization methods use graphs to represent landmarks obtained from both modalities but filtering methods can also be applied. The common strategy being the use of a multi-view-state formulation that includes both sensors measurements in the state vector. Practical implementation of this approach is usually more challenging, as it implicates the fusion of raw data or multi-modal associated features. Nevertheless, it is generally more accurate.

In 2015, inspired by **LOAM** [134] and **DEMO** [132], the same authors extended these techniques to propose a tightly-coupled framework in **V-LOAM** [135]. This method takes camera-based odometry output as motion prior followed by LiDAR odometry, the two modes are tightly combined to represent the surroundings metrically and spatially and to estimate motion with accuracy. The proposed method can handle the temporary absence of texture, as seen in total blackout or whiteout images. Moreover, it can cope with aggressive translation and rotation movements. Real-time performance is achieved by performing camera-based odometry at a high frequency (60Hz) and LiDAR odometry at a low frequency (1Hz). This sequential/parallel design allows the system to maintain a good pose estimation with aggressive motions by bypassing the faulty module. An ICP-based scan to scan refinement step matches point clouds between successive scans to refine motion estimates

and eliminate point cloud distortion induced by camera-based odometry drift. Then, to generate maps continuously, distortion-free point clouds are matched and registered. Even if the authors intentionally chose not to implement loop closure detection to push the boundaries of precise odometry estimate, it achieves a 0.75% of relative position drift.

Inspired by this strategy, **VLO-IMU** [7] proposed integrating reduced IMU data with Camera-LiDAR odometry output in a Kalman Filter to provide a reliable navigation solution. By contrast, this method uses stereo camera odometry with windows processing, bucketing and Bundle Adjustment to reduce growth rate accumulated errors. Moreover, the locally optimized output of camera-based odometry is used as initial transformation for LiDAR odometry. The authors used then Point Cloud Library (PCL) and ICP algorithm for scan matching after eliminating points that are either far (more than 20m), on the ground or exceed 180° Field-of-View. The relevant IMU data that were used are the 2D horizontal acceleration (X and Y-axis) and one 1-D vertical gyroscope (Z-axis). Finally, the roll, pitch, azimuth, velocity and position are refined using Kalman Filter to merge the results of different modules.

Alternatively to using camera-based odometry as motion initial guess for LiDAR odometry or using a Kalman Filter to merge data, another approach was proposed in **TVLO** [106] to combine both measurements. In this framework two maps in different modalities are built and both of them are used to solve the residuals for odometry. Such a tightly-coupled method estimates motion more accurately by adding more geometric constraints. The implementation uses **LOAM** pipeline to generate a voxel map and an **ORB-SLAM2** pipeline to obtain a visual map. The two pipelines are run separately by extracting features individually from each modality and matching them in their respective domain. In the Laser domain, the residuals are geometry metric functions of point-to-plane and point-to-line. In the 2D visual domain, the monocular and stereo projections of the 3D map points form the residuals. At the tracking step, the pose is estimated by minimizing residuals from both modalities using the Levenberg-Marquardt nonlinear optimization method. optimization method like the Levenberg-Marquardt. Afterwards, the pose tracking results are used to update the LiDAR voxel map, and a local Bundle Adjustment is used to update the visual map based on map points. To make the LiDAR voxel map more uniform throughout the surrounding environment, inspired by **Google Cartographer**, this method uses Ceres for loop detection and closure before updating the final map using pose graph optimization. Even if experimental results show that this strategy outperforms state-of-the-art algorithms in accuracy, efforts still need to be made to adapt it for real-time applications.

In a related approach, **LC-LVF** [55] proposed using a new cost-function that considers both scan and image data as constraints for pose graph optimization. First, scan-to-scan or scan-to-map methods are used to estimate the current pose while ORB features are also used for motion estimation and to generate Bag-of-Words. Depending on the scene conditions (featureless, colorful, large scale, etc.) a weighting is performed to balance the two estimated quantities. Pose graph optimization is further carried out using g^2o. Finally Bag-of-Words based approach is applied to detect revisited places. Another contribution of this work is the generated 2.5D map that combines and align both laser planes and image features. This map is used also for fast relocalization but can also be generalized to dynamic objects detection in the future.

Shao et al. proposed **VIL-SLAM** [110] which is based on a tightly-coupled stereo visual inertial LiDAR odometry. The Kanade Lucas Tomasi (KLT) feature tracker is used in the camera-based front-end to track all feature points in previous stereo matches. If the number of detected stereo matches falls below a certain level, Shi-Tomashi Corner detector is applied for features extraction. After computing ORB descriptors on final features, a brute-force stereo matching is performed to produce more stereo matches. This output is then combined with IMU measurements to perform a tightly-coupled fixed-lag smoothing over a pose graph. A LiDAR mapping module is used to align all scanned points to the time of end of scan before carrying out scan-to-map registration. In the loop closure module, point cloud ICP alignment refines the initial loop constraint estimations that follow visual place recognition. To get a globally corrected trajectory and a LiDAR pose correction in real-time, a global pose graph constraining all LiDAR poses is incrementally optimized. These corrections are sent back to LiDAR mapping module for map update and relocalization. Another novelty in this approach is this enhanced loop closure that adds ICP refinement to visual Bag-of-Words.

LIC-Fusion introduced by Zuo et al. [138] tightly combines IMU measurements, extracted LiDAR edge features, as well as sparse FAST visual features, using the Multi-State Constraint Kalman Filter (MSCKF) fusion framework. This work has been recently improved by the same authors in **LIC-Fusion 2.0** [139] through making use of a sliding window based plane-feature tracking approach to efficiently process 3D LiDAR point clouds. An outlier rejection criterion is presented for the proposed plane tracking, enabling robust matching by accounting for the transformation uncertainty between LiDAR frames. The system can accurately represent the uncertainties of LiDAR data, which avoids the discrepancy caused when matching LiDAR scans with ICP.

R²LIVE [70] is the most recent tightly-coupled LiDAR-Inertial-Visual state estimator and mapping. The proposed framework uses a high-rate filter-based odometry and a low-rate factor graph optimization. This multi-sensor system fuses the measurements of camera, LiDAR and inertial sensors within a Kalman filter to achieve real-time performance. The factor graph optimization refines a local map of keyframe poses and visual landmark positions. This approach was tested on several indoor-outdoor datasets using desktop PC (with Intel i7-9700K CPU and 32GB RAM) and onboard computer (with Intel i7-8550u CPU and 8GB).

6 Discussion

6.1 Open Challenges

Our work shows that a clear tendency to integrate multiple sensors for a more accurate and robust SLAM framework has emerged in the last years. However, the more different the sensors are the more challenges are faced to process raw data and to respect exact time alignment. Moreover, spacial transformations can become more complex, which can only increase the algorithmic complexity and limit the optimization possibilities.

In our opinion, merging both modalities and taking full advantage of all the available information by unifying data representation will allow more robust and accurate SLAM systems to emerge. However, this possible solution will have to be more computationally efficient in limited memory hardware. In order to overcome these challenges, we believe that taking hardware specifications into account while designing a tightly-coupled multi-modal SLAM algorithm is a viable research orientation and will significantly improve the overall performance of future SLAM systems. Table 5 summarizes the compares the previously presented coupling strategies while emphasizing each method's benefits and drawbacks.

6.1.1 Map Representation

As discussed in Section 2.4, mapping is an essential part of SLAM and using a dynamic high precision map is critical for autonomous vehicles. However, using either camera or LiDAR, we can represent the environment through multiple map types. There are three main map categories: metric, semantic and hybrid. Each with its own advantages and disadvantages, choosing the right map for the right scenario can affect the global performance [9]. For large-scale environments combining a local detailed map and a global summarized map can solve some real-time-related problems. However, combining different types of maps in order to establish precise maps of outdoor, large and dynamic environments is still a challenge for the upcoming endeavors [9]. On another note, for long term mapping and autonomy, some open questions remain: at what rate should the map be updated? Indeed, because of the quantity and quality of data necessary, the expense of maintaining data-collection systems, and the processing time, map creation remains a complicated operation [10]. Additionally, there is no standarization for the 3D representationf of the map and the detectable patterns that should be used in map building. Other than periodically updating the map, using a cross-modal map representation can be a possible solution to compress heavyweight large-scale maps [82].

6.1.2 Adaptability and Robustness

No SLAM system can now manage all scenarios. In order to perform successfully for a particular circumstance, most systems require substantial parameter adjustment. The data association module in front-end block suffers the most from this. These parameters can also include

Table 5 Summary and comparison between camera-LiDAR coupling strategies

Methods	Description	Benefits	Drawbacks
Tightly-Coupled			
[7, 55, 70, 106, 110, 139]	– Combine the two modalities in one optimal framework – Fusion at data representation or optimization level	– More robust and accurate in challenging scenarios – Takes full advantage of all available data – Multi-level fusion possibilities	– More challenging integration of both modalities – Reduced reliability and increased development cost – Greedy in memory need and computational power
Loosely-Coupled [3, 42, 69, 74, 98, 113, 128, 135]	– Each modality runs separately – Exchanging relevant information in an optimal fashion – Fusion at front end-level	– Easier data association and representation with efficient computation – Can work in dynamic scenes and large scale environments	– Accuracy problems due to matching issues – Minimal data usage and data association dependency – Necessitates robust estimation and optimization to overcome outliers

thresholds that control feature tracking, criteria to insert new keyframe, criteria to close the loop. In order to design an adaptable SLAM system to variant scenarios, additional modules for automatic tuning of parameters need to be incorporated [16]. Additionnaly, in order to enhance an autonomous vehicle perception, appearance-based instead of feature-based methods are preferred, because this will significantly improve long term place recognition and close the loop even between different day and night periods or different seasons. Moreover, mapping with one modality (e.g., LiDAR) and localizing in the same map with a different sensor (e.g., camera) can be interesting, especially if additional techniques such as visual vocabularies [89] or Bag-of-Words [38] are used. These techniques however are not immune to failing (e.g., in case of severe illumination variations) and can be upgraded by using sequence or trajectory matching.

6.1.3 Online and Life-Long Learning

Life-long and online learning is a significant problem that any future long-term SLAM system will be required to solve. SLAM systems are often used in an open environment with uninterrupted observation, where new objects and situations can be met at any time. On the other hand, deep networks are often trained on a limited number of scenarios with a smaller set number of object classes compared to the ones encountered in the real world [97]. A significant challenge is to push the limits of deep networks capabilities in a one-shot or zero-shot scenario (i.e., a new class may have one or even zero training examples) to enable life-long learning for a SLAM system that is constantly moving and observing.

Wofk et al. [127] has successfully estimated depth from a single color image with low latency destined for an embedded system. Additionally, prior information about the environment can provide significant boost to SLAM's accuracy and reliability. This prior information can be geometrical or based on objects and their relationships [99, 117]. Correctly extracting and using this information is a very important challenge since fusing it with geometrical information can bootstrap the SLAM system. In fact, characterizing the uncertainty of estimates derived from deep architectures and designing an end-to-end Deep Neural Network inspired by [20, 76, 77] many solve many problems in feature modeling, data associations, etc. However, in an open world, the autonomous vehicle will encounter an ever changing environment. This will require the SLAM system of the future to learn, adapt and extent its knowledge while operating.

6.1.4 Risk Management

A perfect SLAM system must be both fail-safe and fail-aware. It is not a matter of relocalization or loop closure in this case. The SLAM system must be capable of responding to risk or failure. Its ability should be extended, but not limited, to detecting data corruption and outliers, respecting operational timing, resolving modalities heterogeneity and data redundancy. The design of SLAM system should take into account a risk management module that will assess the risk associated with each sensor due to circumstantial hazards. More frameworks such as [136] should be introduced to enable the system to detect faults in estimation, tracking or data associations. At the same time, an ideal SLAM solution should be able to run on a variety of platforms, regardless of their computing limitations. A difficult task is balancing accuracy, resilience, and limited resources. Adding this challenge to risk assessment, enabling a SLAM system to detect its computational limitations and remove redundant or unnecessary data is an open problem [107].

6.2 Future Orientations

It has become evident that fusion is the future, but the questions remain what should we fuse? Data, based on multiple used sensors? or components, from off-the-shelf such as recently emerged Neural Networks?

6.2.1 Deep Learning and Classical Hybrid Models

The number of researchers focusing on the application of deep learning to SLAM is continuously increasing. Either an end-to-end model [59, 78, 129] or a hybrid model [20, 32, 42] are adopted in literature. Fundamentally, the classic physical and geometrical approaches that govern SLAM will outperform data-driven approaches in untrained scenarios. Moreover, it is difficult to integrate heterogeneous data in an end-to-end network structure. By using a partial concurrent network structure to actualize the fusion of various sensors by accepting multiple sensor data as input and simultaneous processing results as input in the intermediate layer of the network, deep learning capabilities may be leveraged while keeping a theoretically established model. The network can learn the data properties of various sensors and conducts fusion in the intermediate feature layer through a large number of data training sessions. At the same time, the network's depth must account for the quantity of data, the channel number as well as the sampling rate and the frequency of acquisition of various sensor data. Another option is to combine data using the

concept of supervised learning. One sensor can supervise the data of another sensor to enhance it or correct it. For example, Generative Adversarial Networks (GANs) may be used to track the LiDAR point cloud in order to generate more accurate data and remove distortions [103]. Moreover, deep learning based techniques are generally computationally greedy [32, 66, 78]. Therefore, for ground autonomous vehicle applications such models should scale down its computational cost while maintaining acceptable performance. In order to achieve this while balancing the speed and the accuracy of these models, a possibility is to reduce the amount of inputs (either pixels, points or voxels) and the depth of the network [53].

6.2.2 Tightly-Coupled Front-End

A tight-by-design strategy that takes advantage of both sensor modalities does not exit. We propose using one of the fusion strategies to include both sensors and both corresponding algorithms into the final SLAM framework. Such possible fusion solutions can be:

Data in, data out [110]: Feeding raw sensor data into the fusion network, while the output is processed (typically enhanced) raw data taking advantage of both modalities and depending on each one to eliminate the disadvantages.

Data in, feature out [106]: Raw data is combined to create a set of output features independently of the entered data type.

Feature in, feature out [30]: Where the input and the output are both feature vectors. That can also be seen as translation of features to a cross-modal feature.

Feature in, decision out [70, 139]: Tightly coupling the motion estimation module that will be capable of processing either direct data or features from both modalities without distinction in order to estimate the vehicle's motion. An associated challenge would be to increase the corresponding block with combined scan matching and feature matching abilities.

Figure 10 shows a suggested pipeline for a SLAM framework that tightly combines both Camera and LiDAR data in the front-end block.

6.2.3 Tightly-Coupled Back-End

An other way of taking advantage of the multi-modality in SLAM framework would be to implement a back-end block capable of minimizing both residuals while arbitrating between both modalities. We propose to investigate the possibility of sharing constraints obtained from Camera or LiDAR. For instance, rotational errors could be minimized by visually detecting a turn and adding this this information as a semantic constraint to the global back-end. Another example would be measuring distance to an object by LiDAR and forcing the range estimation to reduce the drift.

Such SLAM system will be able to build on the work of Shao et al. in [110] to perform a global pose graph optimization that can incorporate constraints and relative transformation information from both LiDAR and camera data in order to reduce drift.

6.2.4 Data Fusion at Sensor Level

Handling large maps in order to achieve long-term tracking and relocalization requires large memory resources and computational power. Therefore, in order to achieve real-time performance on an embedded SLAM system for autonomous vehicles processed data must be kept to the strict minimum of relevant information. This approach will inevitably lead to designing new hardware-based solutions that can ensure a better management of parallel processing, memory usage and energy efficiency. In fact, developing an efficient autonomous ground vehicle will require designing

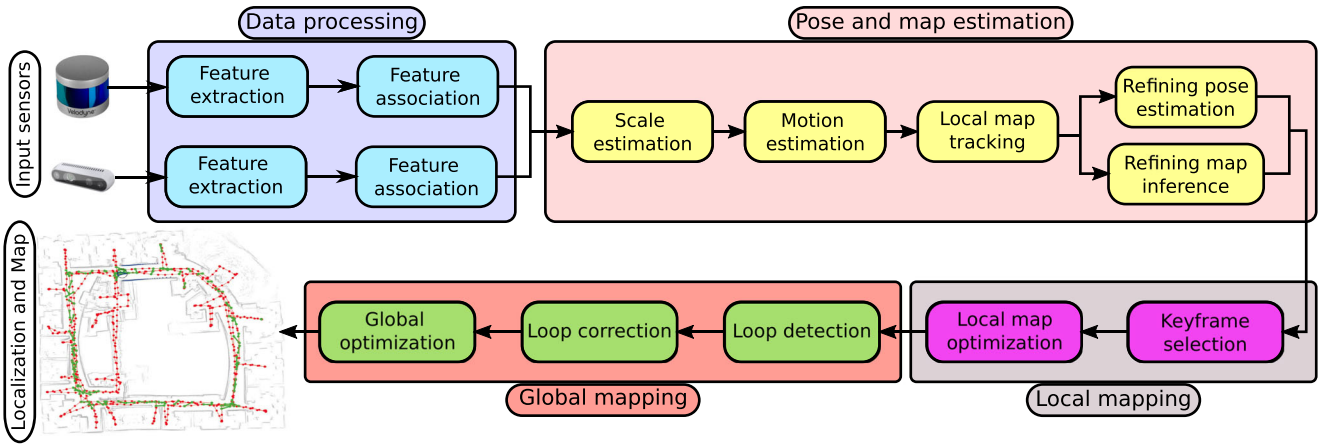


Fig. 10 Proposed SLAM framework

SLAM methods whose memory and computing complexity requirements do not exceed a certain threshold [16]. Using embedded platforms (such as FPGA boards) will solve another critical multi-sensors fusion problem that is data synchronization and pre-processing [124]. By contrast with software-based synchronization techniques that demand additional computing power, hardware-based solutions warrants a unified timing system and a precise time-stamping of the triggered sensors.

6.2.5 Hardware/software Co-Design

Table 4 summarizes the discussed coupled Camera-LiDAR SLAM systems and the hardware architecture that each one of them used. Unfortunately, none of them takes into account hardware constraints associated to embedding such algorithms. In fact, an efficient embedded SLAM must provide high processing capabilities and low energy consumption. The work of Abouzahir et al. in [1] proved that embedded platforms using multi-core and heterogeneous architectures may be used to attain a real-time mono-modal SLAM. Following the same reasoning, future research must take into account a hardware/software co-design methodology when designing a multi-modal SLAM system. Consequently, defining the frequency requirements and the latency restrictions of the whole processing chain must be in line with the targeted platform that carries out the computation [127]. In this sense, CPUs and GPUs provide suitable implementation of complex algorithms than FPGAs. However, FPGAs, generally, lead to better real-time processing thanks to their data-flow processing specifications. They also lead to better power consumption performances. Hence, to achieve an efficient trade-off when addressing the implementation of real-time multi-modal embedded SLAM, the design should rely on heterogeneous hardware architectures combining a CPU and FPGAs. This will reduce the algorithmic complexity by processing small data volumes on CPU at a low frequency, all while larger volumes are efficiently and massively parallelized and processed on a multi-FPGA architecture.

7 Conclusion

In this work we presented Simultaneous Localization and Mapping (SLAM) based on Camera and LiDAR solutions. We first, presented the multiple possible sensors that can be used in the SLAM framework and answered the question to why Camera and LiDAR are the best choice for SLAM.

A brief presentation was formulated to introduce the theory behind SLAM problem, its building blocks and the techniques uses to solve it. We then review the most famous SLAM algorithms that use either Camera data or Laser data then compare the presented algorithms in each modality. Later, we highlight the coupled strategies using both Camera and LiDAR data to perform SLAM and we classify them. We draw special attention to the strengths and weaknesses of each one of them. We finally discuss the future challenges and open problems that still need to be addressed and we present some future research directions for the multi-modal SLAM.

Encouraging results have been obtained in recent research due to the algorithmic maturity that Camera-LiDAR SLAM algorithms have reached. However, designing an end-to-end system embedding these algorithms for autonomous vehicles is far from being an achieved objective given the complexity of such algorithms and the hardware architectures constraints. Hence, a hardware/software co-design approach remains essential and unexplored. Building on this, in our future work, we will study the design of a sensor-fusion-based, tightly-coupled implementation of SLAM in the future. By combining camera frames with LiDAR point clouds while taking into account algorithmic and hardware constraints, we expect to build a robust, low-cost and low-drift end-to-end real-time embedded SLAM system.

Author Contributions All authors contributed to the study conception and methodology. Abdelhafid El Ouardi and Sergio Rodriguez had the idea for the article and supervised the process. Sergio Rodriguez was in charge of providing useful insights to the algorithmic aspects of the discussed SLAM methods. Abdelhafid El Ouardi was in charge of providing useful insights to the hardware aspects of the discussed SLAM systems. Mohammed Chghaf was in charge of synthetizing recent mono-modal and multi-modal SLAM strategies and prepared the first draft of the manuscript. He was in charge of investigation, literature search and data analysis. Sergio Rodriguez and Abdelhafid El Ouardi commented on previous versions of the manuscript and critically revised the work. All authors read and approved the final manuscript.

Funding This work was supported by the French Ministry of Higher Education, Research and Innovation.

Code Availability Not applicable

Declarations

Ethics approval and consent to participate Not applicable

Consent for Publication Not applicable

Conflicts of interest/Competing interests The authors declare that they have no known conflicts or competing interests that could have appeared to influence the work reported in this paper.

References

1. Abouzahir, M., Elouardi, A., Latif, R., Bouaziz, S., algorithms, A.T.: Embedding slam has it come of age? Robot. Auton. Syst. **100**, 14–26 (2018)

2. Agarwal, S., Mierle, K., et al.: Ceres solver. http://ceres-solver.org
3. Andresen, L., Brandemuehl, A., Hönger, A., Kuan, B., Vödisch, N., Blum, H., Reijgwart, V., Bernreiter, L., Schaupp, L., Chung, J.J., et al: Accurate mapping and planning for autonomous racing. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4743–4749 (2020)
4. Andrew, A.M.: Multiple view geometry in computer vision Kybernetes (2001)
5. Arandjelovic, R., Gronat, P., Torii, A., Pajdla, T., Josef, S.: NetVLAD: CNN architecture for weakly supervised place recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 5297–5307 (2016)
6. Arandjelovic, R., Zisserman, A.: All about VLAD. In: Proceedings of the IEEE conference on Computer Vision and Pattern Recognition, pp. 1578–1585 (2013)
7. Sarvrood, Y.B., Hosseinyalamdary, S., Gao, Y.: Visual-liDAR odometry aided by reduced IMU. ISPRS Int. J. Geo-Inform. **5**(1), 3 (2016)
8. Behley, J., Stachniss, C.: Efficient surfel-based slam using 3d laser range data in urban environments. In: Robotics: Science and Systems 2018 (2018)
9. Bernuy, F., Ruiz-del Solar, J.: Topological semantic mapping and localization in urban road scenarios. J. Intell. Robot. Syst. **92**(1), 19–32 (2018)
10. Berrio, J.S., Worrall, S., Shan, M., Nebot, E.: Long-term map maintenance pipeline for autonomous vehicles. arXiv:2008.12449 (2020)
11. Besl, P.J., McKay, N.D.: Method for registration of 3-d shapes. In: Sensor Fusion IV: Control Paradigms and Data Structures, vol. 1611, pp. 586–606. International Society for Optics and Photonics (1992)
12. Biber, P., Straßer, W.: The normal distributions transform A new approach to laser scan matching. In: Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)(Cat. No. 03CH37453), vol. 3, pp. 2743–2748. IEEE (2003)
13. Blanco, J.-L., Fernández-Madrigal, J.-A., Gonzalez, J.: Toward a unified Bayesian approach to hybrid metric–topological SLAM. IEEE Trans. Robot. **24**(2), 259–270 (2008)
14. Borenstein, J., Everett, H.R., Feng, L., Wehe, D.: Mobile robot positioning: Sensors and techniques. J. Robot. Syst. **14**(4), 231–249 (1997)
15. Bowman, S.L., Atanasov, N., Daniilidis, K., Pappas, G.J.: Probabilistic data association for semantic SLAM. In: 2017 IEEE International Conference on Robotics and Automation (ICRA), pp. 1722–1729. IEEE (2017)
16. Cadena, C., Carlone, L., Carrillo, H., Latif, Y., Scaramuzza, D., Neira, J., Reid, I., Leonard, J.J.: Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. IEEE Trans. Robot. **32**(6), 1309–1332 (2016)
17. Campos, C., Elvira, R., Gómez Rodríguez, J.J., Montiel, J., Tardós, J.D.: ORB-SLAM3: An accurate open-source library for visual, visual-inertial and multi-map SLAM. arXiv:2007.11898 (2020)
18. Cao, F., Zhuang, Y., Zhang, H., Wang, W.: Robust place recognition and loop closing in laser-based SLAM for ugvs in urban environments. IEEE Sensors J. **18**(10), 4242–4252 (2018)
19. Censi, A.: An ICP variant using a point-to-line metric. In: 2008 IEEE International Conference on Robotics and Automation, pp. 19–25. IEEE (2008)
20. Chen, X., Milioto, A., Palazzolo, E., Giguere, P., Behley, J., Stachniss, C.: SuMa++: Efficient LiDAR-based semantic slam. In: 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4530–4537. IEEE (2019)
21. Concha, A., Civera, J.: DPPTAM: Dense piecewise planar tracking and mapping from a monocular sequence. In: 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 5686–5693. IEEE (2015)
22. Cvišić, I., Ćesić, J., Marković, I., Petrović, I.: SOFT-SLAM: Computationally efficient stereo visual simultaneous localization and mapping for autonomous unmanned aerial vehicles. J. Field Robot. **35**(4), 578–595 (2018)
23. Das, A., Waslander, S.L.: Scan registration with multi-scale k-means normal distributions transform. In: 2012 IEEE/RSJ International Conference On Intelligent Robots and Systems, pp. 2705–2710. IEEE (2012)
24. Davison, A.J.: Real-time simultaneous localisation and mapping with a single camera. In: IEEE International Conference on Computer Vision, vol. 3, pp. 1403–1403. IEEE Computer Society (2003)
25. Davison, A.J., Reid, I.D., Molton, N.D., Olivier, S.: MonoSLAM: Real-time single camera SLAM. IEEE Trans. Pattern Anal. Mach. Intell. **29**(6), 1052–1067 (2007)
26. De Croce, M., Pire, T., Bergero, F.: DS-PTAM: Distributed stereo parallel tracking and mapping SLAM system. J. Intell. Robot. Syst. **95**(2), 365–377 (2019)
27. Debeunne, C., Vivet, D.: A review of visual-lidar fusion based simultaneous localization and mapping. Sensors **20**(7), 2068 (2020)
28. Dellaert, F.: Factor graphs and GTSAM: A handson introduction. Technical report, Georgia Institute of Technology (2012)
29. Deschaud, J.-E.: IMLS-SLAM: Scan-to-model matching based on 3d data. In: 2018 IEEE International Conference on Robotics and Automation (ICRA), pp. 2480–2485. IEEE (2018)
30. Ding, X., Wang, Y., Xiong, R., Li, D., Li, T., Yin, H., Zhao, L.: Persistent stereo visual localization on cross-modal invariant map. IEEE Trans. Intell. Transp. Syst. **21**(11), 4646–4658 (2019)
31. Dissanayake, G.M.W.M., Newman, P., Clark, S., Durrant-Whyte, H.F., Csorba, M.: A solution to the simultaneous localization and map building (SLAM) problem. IEEE Trans. Robot. Autom. **17**(3), 229–241 (2001)
32. Dubé, R., Cramariuc, A., Dugas, D., Nieto, J., Siegwart, R., Cadena, C.: SegMap: 3d segment mapping using data-driven descriptors. arXiv:1804.09557 (2018)
33. Einhorn, E., Gross, H.-M.: Generic NDT mapping in dynamic environments and its application for lifelong SLAM. Robot. Auton. Syst. **69**, 28–39 (2015)
34. Engel, J., Koltun, V., Cremers, D.: Direct sparse odometry. IEEE Trans. Pattern Anal. Mach. Intell. **40**(3), 611–625 (2017)
35. Engel, J., Schöps, T., Cremers, D.: LSD-SLAM: Large-scale direct monocular SLAM. In: European Conference on Computer Vision, pp. 834–849. Springer (2014)
36. Forster, C., Zhang, Z., Gassner, M., Werlberger, M., Davide, S.: SVO: Semidirect Visual odometry for monocular and multicamera systems. IEEE Trans. Robot. **33**(2), 249–265 (2016)
37. Fuentes-Pacheco, J., Ruiz-Ascencio, J., Rendón-Mancha, J.M.: Visual simultaneous localization and mapping: a survey. Artif. Intell. Rev. **43**(1), 55–81 (2015)
38. Gálvez-López, D., Tardos, J.D.: Bags of binary words for fast place recognition in image sequences. IEEE Trans. Robot. **28**(5), 1188–1197 (2012)
39. Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? the KITTI vision benchmark suite. In: Conference on Computer Vision and Pattern Recognition (CVPR) (2012)
40. Geiger, A., Ziegler, J., Stiller, C.: StereoScan: Dense 3d reconstruction in real-time. In: Intelligent Vehicles Symposium (IV) (2011)

41. Gong, Z., Ying, R., Wen, F., Qian, J., Liu, P.: Tightly coupled integration of GNSS and vision SLAM using 10-DoF optimization on manifold. IEEE Sensors J. **19**(24), 12105–12117 (2019)
42. Graeter, J., Wilczynski, A., Lauer, M.: LIMO: Lidar-monocular visual odometry. In: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 7872–7879. IEEE (2018)
43. Grisetti, G., Kümmerle, R., Stachniss, C., Burgard, W.: A tutorial on graph-based SLAM. IEEE Intell. Transp. Syst. Mag. **2**(4), 31–43 (2010)
44. Grisetti, G., Stachniss, C., Burgard, W.: Improved techniques for grid mapping with rao-blackwellized particle filters. IEEE Trans. Robot. **23**(1), 34–46 (2007)
45. Guo, Y., Sohel, F., Bennamoun, M., Lu, M., Wan, J.: Rotational projection statistics for 3d local surface description and object recognition. Int. J. Comput. Vis. **105**(1), 63–86 (2013)
46. He, K., Gkioxari, G., DollPár, P., Girshick, R.: Mask r-CNN. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 2961–2969 (2017)
47. Henry, P., Krainin, M., Herbst, E., Ren, X., Fox, D.: RGB-D mapping: Using depth cameras for dense 3d modeling of indoor environments. In: Experimental Robotics, pp. 477–491. Springer (2014)
48. Hess, W., Kohler, D., Rapp, H., Andor, D.: Real-time loop closure in 2d liDAR SLAM. In: 2016 IEEE International Conference on Robotics and Automation (ICRA), pp. 1271–1278. IEEE (2016)
49. Hong, Z., Petillot, Y., Sen, W.: RadarSLAM: Radar based large-scale SLAM, in all weathers. arXiv:2005.02198 (2020)
50. Hornung, A., Wurm, K.M., Bennewitz, M., Stachniss, C., Burgard, W.: OctoMap: An efficient probabilistic 3d mapping framework based on octrees. Autonom Rob **34**(3), 189–206 (2013)
51. Houseago, C., Bloesch, M., Leutenegger, S.: KO-Fusion: dense visual SLAM with tightly-coupled kinematic and odometric tracking. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 4054–4060. IEEE (2019)
52. Hyun, E., Jin, Y.-S., Lee, J.-H.: Moving and stationary target detection scheme using coherent integration and subtraction for automotive fmcw radar systems. In: 2017 IEEE Radar Conference (RadarConf), pp. 0476–0481. IEEE (2017)
53. Iandola, F.N., Han, S., Moskewicz, M.W., Ashraf, K., Dally, W.J., Keutzer, K.: SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 mb model size. arXiv:1602.07360 (2016)
54. Ji, K., Chen, H., Di, H., Gong, J., Xiong, G., Qi, J., Yi, T.: CPFG-SLAM: a robust simultaneous localization and mapping based on LIDAR in off-road environment. In: 2018 IEEE Intelligent Vehicles Symposium (IV), pp. 650–655. IEEE (2018)
55. Jiang, G., Yin, L., Jin, S., Tian, C., Ma, X., Ou, Y.: A simultaneous localization and mapping (SLAM) framework for 2.5 d map building based on low-cost liDAR and vision fusion. Appl. Sci. **9**(10), 2105 (2019)
56. Kaess, M., Ranganathan, A., Frank, D.: ISAM: Incremental smoothing and mapping. IEEE Trans. Robot. **24**(6), 1365–1378 (2008)
57. Kim, G., Kim, A.: Scan context: Egocentric spatial descriptor for place recognition within 3D point cloud map. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems Madrid (2018)
58. Kim, H., Leutenegger, S., Davison, A.J.: Real-time 3d reconstruction and 6-dof tracking with an event camera. In: European Conference on Computer Vision, pp. 349–364. Springer (2016)
59. Kim, U.-H., Kim, S., Jong-Hwan, K.: SimVODIS: Simultaneous visual odometry, object detection, and instance segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence (2020)
60. Klein, G., Murray, D.: Parallel tracking and mapping for small AR workspaces. In: 2007 6th IEEE and ACM International Symposium on Mixed and Augmented Reality, pp. 225–234. IEEE (2007)
61. Klein, G., Murray, D.: Parallel tracking and mapping on a camera phone. In: 2009 8th IEEE International Symposium on Mixed and Augmented Reality, pp. 83–86. IEEE (2009)
62. Kohlbrecher, S., Meyer, J., von Stryk, O., Klingauf, U.: A flexible and scalable SLAM system with full 3d motion estimation. In: Proc. IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR). IEEE (2011)
63. Konolige, K., Grisetti, G., Kümmerle, R., Burgard, W., Limketkai, B., Vincent, R.: Efficient sparse pose adjustment for 2d mapping. In: 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 22–29. IEEE (2010)
64. Kümmerle, R., Grisetti, G., Strasdat, H., Konolige, K., Burgard, W.: g2o: A general framework for graph optimization. In: 2011 IEEE International Conference on Robotics and Automation, pp. 3607–3613. IEEE (2011)
65. Laidlow, T., Bloesch, M., Li, W., Leutenegger, S.: Dense RGB-D-Inertial SLAM with map deformations. In: 2017 IEEE/RSJ International Conference On Intelligent Robots and Systems (IROS), pp. 6741–6748. IEEE (2017)
66. Li, Q., Chen, S., Wang, C., Li, X., Wen, C., Cheng, M., Li, J.: LO-Net: Deep real-time liDAR odometry. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8473–8482 (2019)
67. Li, R., Wang, S., Gu, D.: DeepSLAM: A robust monocular SLAM system with unsupervised deep learning. IEEE Trans. Ind. Electron. **68**(4), 3577–3587 (2020)
68. Li, Y., Ushiku, Y., Tatsuya, H.: Pose graph optimization for unsupervised monocular visual odometry. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 5439–5445. IEEE (2019)
69. Liang, X., Chen, H., Li, Y., Liu, Y.: Visual laser-SLAM in large-scale indoor environments. In: 2016 IEEE International Conference on Robotics and Biomimetics (ROBIO), pp. 19–24. IEEE (2016)
70. Lin, J., Zheng, C., Xu, W., Fu, Z.: R2LIVE: A robust, real-time, LiDAR-inertial-visual tightly-coupled state estimator and mapping, arXiv:2102.12400 (2021)
71. Liu, Q., Duan, F.: Fast and consistent matching for landmark-based place recognition. Journal of Intelligent & Robotic Systems, 1–14 (2020)
72. Liu, Y., Yang, D., Li, J., Gu, Y., Pi, J., Zhang, X.: Stereo visual-inertial SLAM with points and lines. IEEE Access **6**, 69381–69392 (2018)
73. Liu, Y., Thrun, S.: Results for outdoor-SLAM using sparse extended information filters. In: 2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422), vol. 1, pp. 1227–1233. IEEE (2003)
74. López, E., García, S., Barea, R., Bergasa, L.M., Molinos, E.J., Arroyo, R., Romera, E., Pardo, S.: A multi-sensorial simultaneous localization and mapping (SLAM) system for low-cost micro aerial vehicles in GPS-denied environments. Sensors **17**(4), 802 (2017)
75. Low, K.-L.: Linear least-squares optimization for point-to-plane ICP surface registration. Chapel Hill, University of North Carolina **4**(10), 1–3 (2004)

76. Lu, W., Wan, G., Zhou, Y., Fu, X., Yuan, P., Song, S.: DeepICP: An end-to-end deep neural network for 3d point cloud registration. arXiv:1905.04153 (2019)
77. Lu, W., Wan, G., Zhou, Y., Fu, X., Yuan, P., Shiyu, S.: DeepVCP: An end-to-end deep neural network for point cloud registration. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 12–21 (2019)
78. Lu, W., Zhou, Y., Wan, G., Hou, S., Shiyu, S.: L3-Net: Towards Learning based liDAR localization for autonomous driving. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 6389–6398 (2019)
79. Magnusson, M., Lilienthal, A., Duckett, T.: Scan registration for autonomous mining vehicles using 3d-NDT. J Field Robot **24**(10), 803–827 (2007)
80. McCormac, J., Clark, R., Bloesch, M., Davison, A., Leutenegger, S.: Fusion++: Volumetric object-level SLAM. In: 2018 International Conference on 3D Vision (3DV), pp. 32–41. IEEE (2018)
81. Mendes, E., Koch, P., Lacroix, S.: pose-graph SLAM. In: 2016 Icp-based IEEE International Symposium On Safety, Security, and Rescue Robotics (SSRR), pp. 195–200. IEEE (2016)
82. Mithun, N.C., Sikka, K., Chiu, H.-P., Samarasekera, S., Rakesh, K.: RGB2LIDAR: Towards solving large-scale cross-modal visual localization. In: Proceedings of the 28th ACM International Conference on Multimedia, pp. 934–954 (2020)
83. Muja, M., Lowe, D.G.: Fast approximate nearest neighbors with automatic algorithm configuration. VISAPP (1) **2**(331-340), 2 (2009)
84. Munoz-Salinas, R., Rafael, M.-C.: UcoSLAM: Simultaneous localization and mapping by fusion of keypoints and squared planar markers. Pattern Recogn. **101**, 107193 (2020)
85. Mur-Artal, R., Martinez Montiel, J.M., Tardos, J.D.: ORB-SLAM: A versatile and accurate monocular SLAM system. IEEE Trans. Robot. **31**(5), 1147–1163 (2015)
86. Mur-Artal, R., Tardós, J.D.: ORB-SLAM2:An open-source SLAM system for monocular, stereo, and rgb-d cameras. IEEE Trans. Robot. **33**(5), 1255–1262 (2017)
87. Newcombe, R.A., Izadi, S., Hilliges, O., Molyneaux, D., Kim, D., Davison, A.J., Kohi, P., Shotton, J., Hodges, S., Fitzgibbon, A.: KinectFusion: Real-time dense surface mapping and tracking. In: 2011 10th IEEE International Symposium on Mixed and Augmented Reality, pp. 127–136 (2011)
88. Newcombe, R.A., Lovegrove, S.J., Davison, A.J.: DTAM: Dense tracking and mapping in real-time. In: 2011 International Conference on Computer Vision, pp. 2320–2327. IEEE (2011)
89. Newman, P., Cole, D., Ho, K.: Outdoor SLAM using visual appearance and laser ranging. In: 2006 Proceedings 2006 IEEE International Conference on Robotics and Automation ICRA 2006., pp. 1180–1187. IEEE (2006)
90. Bowden, R., Kaygusuz, N., Mendez, O.: MDN-VO: Estimating visual odometry with confidence. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (2021)
91. Pascoe, G., Maddern, W., Tanner, M., Piniés, P., Paul, N.: NID-SLAM: Robust Monocular SLAM using normalised information distance. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1435–1444 (2017)
92. Pfrommer, B., Kostas, D.: TagSLAM: Robust SLAM, with fiducial markers. arXiv:1910.00679 (2019)
93. Pire, T., Fischer, T., Civera, J., De Cristóforis, P., Berlles, J.J.: Stereo parallel tracking and mapping for robot localization. In: 2015 Stereo parallel IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 1373–1378. IEEE (2015)
94. Polok, L., Ila, V., Solony, M., Smrz, P., Zemcik, P.: Incremental block cholesky factorization for nonlinear least squares in robotics. In: Robotics: Science and Systems, pp. 328–336 (2013)
95. Prophet, R., Li, G., Sturm, C., Vossiek, M.: Semantic segmentation on automotive radar maps. In: 2019 IEEE Intelligent Vehicles Symposium (IV), pp. 756–763. IEEE (2019)
96. Qayyum, U., Ahsan, Q., Mahmood, Z.: IMU aided RGB-D SLAM. In: 2017 14th International Bhurban Conference on Applied Sciences and Technology (IBCAST), pp. 337–341. IEEE (2017)
97. Qi, C.R., Su, H., Mo, K., Guibas, L.J.: PointNet: Deep learning on point sets for 3d classification and segmentation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 652–660 (2017)
98. Radmanesh, R., Wang, Z., Chipade, V.S., Tsechpenakis, G., Panagou, D.: LIV-LAM: LiDAR and visual localization and mapping. In: 2020 American Control Conference (ACC), pp. 659–664. IEEE (2020)
99. Rebecq, H., Horstschäfer, T., Gallego, G., Davide, S.: EVO: A geometric approach to event-based 6-DOF parallel tracking and mapping in real-time. IEEE Robot. Autom. Lett. **2**(2), 593–600 (2016)
100. Redmon, J., Ali, F.: YOLOV3: An incremental improvement. arXiv:1804.02767 (2018)
101. Rusu, R.B., Blodow, N., Beetz, M.: Fast point feature histograms (FPFH) for 3d registration. In: 2009 Fast Point Feature IEEE International Conference on Robotics and Automation, pp. 3212–3217. IEEE (2009)
102. Salas-Moreno, R.F., Glocken, B., Kelly, P.HJ., Davison, A.J.: Dense planar SLAM. In: 2014 Dense IEEE International Symposium on Mixed and Augmented Reality (ISMAR), pp. 157–164. IEEE (2014)
103. Sallab, A.E., Sobh, I., Zahran, M., Essam, N.: Lidar sensor modeling and data augmentation with gans for autonomous driving, arXiv:1905.07290 (2019)
104. Schuster, F., Keller, C.G., Rapp, M., Haueis, M., Curio, C.: SLAM using graph optimization. In: 2016 Landmark based IEEE 19th International Conference on Intelligent Transportation Systems (ITSC), pp. 2559–2564. IEEE (2016)
105. Segal, A., Haehnel, D., Thrun, S.: Generalized-ICP. In: Robotics: Science and Systems, vol. 2, p. 435, Seattle (2009)
106. Seo, Y., Chou, C.-c.: A tight coupling of vision-liDAR measurements for an effective odometry. IEEE (2019)
107. Servières, M., Renaudin, V., Dupuis, A., Antigny, N.: Visual and visual-inertial SLAM: State of the art, classification, and experimental benchmarking. Journal of Sensors, 2021 (2021)
108. Shan, T., Englot, B.: LeGO-LOAM: Lightweight and ground-optimized LiDAR odometry and mapping on variable terrain. In: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4758–4765. IEEE (2018)
109. Shan, T., Englot, B., Meyers, D., Wang, W., Ratti, C., Rus, D.: LIO-SAM: Tightly-coupled LiDAR inertial odometry via smoothing and mapping. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 5135–5142. IEEE (2020)
110. Shao, W., Vijayarangan, S., Li, C., Kantor, G.: Stereo visual inertial LiDAR, simultaneous localization and mapping, arXiv:1902.10741 (2019)
111. Shin, H., Kim, D., Kwon, Y., Kim, Y.: Illusion and dazzle: Adversarial optical channel exploits against lidars for automotive applications. In: International Conference on Cryptographic Hardware and Embedded Systems, pp. 445–467. Springer (2017)

112. Shin, Y.-S., Park, Y.S., Kim, A.: Direct visual SLAM using sparse depth for camera- LiDAR system. In: 2018 Direct IEEE International Conference on Robotics and Automation (ICRA), pp. 5144–5151. IEEE (2018)
113. Shin, Y.-S., Park, Y.S., Kim, A.: DVL-SLAM: Sparse depth enhanced direct visual-liDAR SLAM. Auton. Robot. **44**(2), 115–130 (2020)
114. Song, H., Shin, H.-C.: Classification and spectral mapping of stationary and moving objects in road environments using fmcw radar. IEEE Access **8**, 22955–22963 (2020)
115. Steder, B., Rusu, R.B., Konolige, K., Burgard, W.: NARF: 3d range image features for object recognition. In: Workshop on Defining and Solving Realistic Perception Problems in Personal Robotics at the IEEE/RSJ. Int. Conf. on Intelligent Robots and Systems (IROS), vol. 44 (2010)
116. Strasdat, H., Montiel, J., Davison, A.J.: Visual SLAM: Why filter? Image Vis. Comput. **30**(2), 65–77 (2012)
117. Sünderhauf, N., Pham, T.T., Latif, Y., Milford, M., Reid, I.: Meaningful maps with object-oriented semantic mapping. In: 2017 IEEE/RSJ International Conference On Intelligent Robots and Systems (IROS), pp. 5079–5085. IEEE (2017)
118. Szeliski, R.: Computer Vision: Algorithms and Applications. Springer Science & Business Media (2010)
119. Taketomi, T., Uchiyama, H., Ikeda, S.: Visual SLAM algorithms: a survey from 2010 to 2016. IPSJ Trans. Comput. Vis. Applic. **9**(1), 1–11 (2017)
120. Tam, G.KL., Cheng, Z.-Q., Lai, Y.-K., Langbein, F.C., Liu, Y., Marshall, D., Martin, R.R., Sun, X.-F., Rosin, P.L.: Registration of 3d point clouds and meshes: a survey from rigid to nonrigid. IEEE Trans. Visual. Comput. Graph. **19**(7), 1199–1217 (2012)
121. Tian, Y., Suwoyo, H., Wang, W., Mbemba, D., Li, L.: An AEKF-SLAM algorithm with recursive noise statistic based on MLE and EM. J Intelli Robot Syst **97**(2), 339–355 (2020)
122. Uy, M.A., Lee, H.G.: pointnetVLAD: Deep point cloud based retrieval for large-scale place recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4470–4479 (2018)
123. Vidal, A.R., Rebecq, H., Horstschaefer, T., Scaramuzza, D.: Ultimate SLAM? Combining events, images, and imu for robust visual slam in hdr and high-speed scenarios. IEEE Robot. Autom. Lett. **3**(2), 994–1001 (2018)
124. Wan, Z., Yu, B., Li, T.Y., Tang, J., Zhu, Y., Yu, W., Raychowdhury, A., Liu, S.: A survey of fpga-based robotic computing. IEEE Circ. Syst. Mag. **21**(2), 48–74 (2021)
125. Wang, R., Schworer, M., Daniel, C.: Stereo DSO: Large-scale direct sparse visual odometry with stereo cameras. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 3903–3911 (2017)
126. Wang, Y., Shi, T., Yun, P., Tai, L., Ming, L.: PointSeg: Real-time semantic segmentation based on 3d LiDAR, point cloud. arXiv:1807.06288 (2018)
127. Wofk, D., Ma, F., Yang, T.-J., Karaman, S., Sze, V.: FastDepth: Fast monocular depth estimation on embedded systems. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 6101–6108. IEEE (2019)
128. Yan, M., Wang, J., Li, J., Zhang, C.: Loose coupling visual-LiDAR odometry by combining VISO2 and LOAM. In: 2017 36th Chinese Control Conference (CCC), pp. 6841–6846. IEEE (2017)
129. Yang, N., von Stumberg, L., Wang, R., Daniel, C.: D3VO: Deep depth, deep pose and deep uncertainty for monocular visual odometry. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 1281–1292 (2020)
130. Yu, W., Amigoni, F.: Standard for robot map data representation for navigation. In: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 3–4 (2014)
131. Zhang, G., Liu, H., Dong, Z., Jia, J., Wong, T.-T., Bao, H.: Efficient non-consecutive feature tracking for robust structure-from-motion. IEEE Trans. Image Process. **25**(12), 5957–5970 (2016)
132. Ji, Z., Kaess, M., Singh, S.: Real-time depth enhanced monocular odometry. In: 2014 IEEE/RSJ International Conference On Intelligent Robots and Systems, p. 2014. IEEE (2014)
133. Ji, Z., Kaess, M., Singh, S.: A real-time method for depth enhanced visual odometry. Auton. Robot. **41**(1), 31–43 (2017)
134. Ji, Z., Sanjiv, S.: LOAM: Lidar Odometry and mapping in real-time. In: Robotics: Science and Systems, vol. 2 (2014)
135. Zhang, J., Singh, S.: Visual-LiDAR odometry and mapping: Low-drift, robust, and fast. In: 2015 IEEE International Conference on Robotics and Automation (ICRA), pp. 2174–2181. IEEE (2015)
136. Zheng, X., Huang, B., Ni, D., Xu, Q.: A novel intelligent vehicle risk assessment method combined with multi-sensor fusion in dense traffic environment. Journal of Intelligent and Connected Vehicles (2018)
137. Zhou, Y., Gallego, G., Shen, S.: Event-based stereo visual odometry, arXiv:2007.15548 (2020)
138. Zuo, X., Geneva, P., Lee, W., Liu, Y., Huang, G.: LIC-Fusion: LiDAR-Inertial-Camera odometry, arXiv:1909.04102 (2019)
139. Zuo, X., Yang, Y., Geneva, P., Lv, J., Liu, Y., Huang, G., Pollefeys, M.: LIC-Fusion, 2.0: LiDAR-inertial-camera odometry with sliding-window plane-feature tracking, arXiv:2008.07196 (2020)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Mohammed Chghaf received the M.S. degree in electrical engineering in 2015 (specializing in automation and industrial computing) from Mohammadia School of Engineers, Rabat, Morocco. He received the M.S. degree in embedded systems and data processing in 2019 from Paris-Saclay University, Orsay, France. He is currently pursuing the Ph.D. degree with SATIE laboratory, Paris-Saclay University. His research activities are focused on multi-modal embedded SLAM systems for autonomous vehicles.

Sergio Rodriguez received his M.S. and Ph.D. degrees from University of Technology of Compiègne, France, in 2007 and 2009, respectively. Since 2011, he is an Associate Professor of the Paris-Sud University. His research activities are focused on dynamic scene analysis through multimodal perception intended to enhance Intelligent Transportation Systems applications.

Abdelhafid El Ouardi received the M.S. degrees from Pierre and Marie Curie University in 2001 and the Ph.D. degree in Electronics from Paris-Sud University in 2005. He worked at Henri Poincaré University, Nancy, as a researcher in 2005–2006. He is currently an Associate Prof at Paris-Sud University, France. In Embedded Systems team of SATIE lab, his research interests include Hardware-Software co-design, evaluation and instrumentation of embedded systems, design of smart architectures for image and signal processing, SLAM applications.